

变物性与收缩边界下圆柱药材的热湿迁移建模

摘要

针对圆柱形药材烘干中环境边界时变、内部温湿分布不均、物性非线性及几何收缩相互影响的问题，本文构建统一的变物性径向热湿有限体积框架。以 PCHIP 重构附件中的环境量和半径曲线，以 Fourier–Fick 方程描述内部传递，采用节点中心有限体积法与全隐式格式保证离散守恒和长时计算稳定，并通过 Picard 迭代处理状态相关物性；进一步利用固定域变换统一描述收缩边界。

针对问题一：在固定半径和给定物性下，建立常物性 **Fourier–Fick 基准模型**，将时变环境量作为双 Robin 边界输入并逐秒求解。1800 s 时中心与表面温度分别为 33.577 °C 和 36.786 °C，水分浓度分别为 2.550 和 1.510 kg/kg，表明预热初期水分主要从表层逸出，内部仍接近初始状态。

针对问题二：在基准模型上引入 $\rho(C)$, $c_p(C)$, $k(C)$ 与 $D(T, C)$ ，构造**温度–水分状态耦合的非线性扩散模型**，每个时间层交替更新温度、扩散系数和水分场。3 h 时中心与表面温度达到 49.849 °C 和 49.967 °C，水分浓度降至 1.766 和 1.008 kg/kg；此时径向温差已很小，而水分梯度仍然明显，后续过程主要受内部传质控制。

针对问题三：将问题二模型延伸至长期干燥，以 $\max_r C(r, t) \leq 0.15$ kg/kg 定义全截面达标，并对跨越阈值的相邻时间层实施线性事件插值。**全域首达检测得到干燥时间为 57.169 h**；达标瞬间中心和表面水分浓度分别为 0.150 和 0.053 kg/kg。全程中心均为最大值，说明仅依据表面含水率停机将低估内部干燥时间。

针对问题四：令 $\xi = r/R(t)$ ，补充均匀材料收缩及干骨架守恒假设，使材料速度与网格速度相同，在固定域中求解相对输运为零的扩散方程；同时设置附件四物性下的固定半径反事实模型以隔离几何效应。**移动边界模型得到干燥时间为 50.821 h**，达标半径为 1.200 cm；同物性固定半径方案需 129.093 h，故收缩使时长缩短 60.6%，其主导机制是径向扩散距离持续减小。

采用 161 个表面加密节点和 1 s 步长终算。节点加密至 321 个及步长由 2 s 减为 1 s 时，问题三、四达标时间的相对变化均小于 2×10^{-5} ；长期二维核验进一步检查全域控制位置，离散质量与有效热方程平衡残差均接近舍入误差水平。灵敏度采用一致步长和明确的温度增量定义，结果表明环境温度是主要外部控制量。模型结论依赖给定边界与材料收缩假设，可作为多孔生物材料干燥预测的基线。

关键字： 药材烘干 Fourier–Fick 模型 有限体积法 移动边界 首达时间

一、问题重述

1.1 问题背景

药材烘干的目标并非简单地使表面水分降低，而是在不过度加热和不过度干燥表层的前提下，使药材内部所有位置均达到规定含水标准。烘房温度升高后，热量由外向内传递，表层水分同时在内部浓度梯度和表面对流传质的共同作用下向外迁移。由于传热和传质的特征时间不同，药材内部会长期存在“表面已干、中心仍湿”的空间不均匀现象。因此，只记录环境状态或表面状态不能直接判断整根药材是否完成烘干，必须建立能够描述内部时空分布的非稳态模型。

题目研究对象为长度 25 cm、初始半径 2 cm 的圆柱形药材，其初始温度为 28 °C，初始水分浓度为 2.55 kg/kg。附件一给出前 4 h 烘房温度和水分浓度的离散测量值，附件二给出 72 h 内药材半径的变化记录。四个问题依次加入状态相关物性、全截面达标约束和几何收缩效应，实质上构成由固定区域线性问题向移动区域非线性问题逐层推进的建模任务。

1.2 问题提出

根据所给几何、初始条件、环境序列和物性关系，需要获得药材温度、水分浓度随时间及径向位置的分布，并确定内部最难干燥位置首次满足品质阈值的时刻。为使四问结果具有统一口径，还需要明确环境数据的插值方式、表面传递边界、空间离散方法、终止事件以及收缩后“固定物理位置”的有效范围。

问题一：在半径和主要热物性保持不变的基准情形下，处理时变环境与药材表面的热、质交换，计算预热阶段 0–1800 s 内的径向温度和水分浓度，并形成每 1 s、每 0.1 cm 的完整结果。该问用于建立后续各问共享的控制方程、边界条件和数值离散框架。

问题二：在问题一基础上，将密度、比热容、导热系数及水分扩散系数改写为温度和水分浓度的函数。模型由常系数扩散方程转为状态耦合的非线性方程，需要求得完整烘干过程的温湿场，正文展示前 3 h 的代表结果，并分析物性变化如何影响中心与表面的响应差异。

问题三：沿用问题二的变物性模型，将计算时域延长至药材内部所有位置的水分浓度均不超过 0.15 kg/kg。该问的关键不是某个离散输出时刻是否达标，而是确定全空间最大水分浓度首次跨越阈值的连续时间，并给出每 60 s 的完整结果与每 6 h 的典型剖面。

问题四：进一步考虑药材半径随时间收缩，并使用附件四给出的另一组状态相关物性。计算域随时间变化，固定半径采样点还可能在后期落到药材外部。因此，需要建立

移动边界模型，重新确定达标时长，规范动态表面及固定物理位置的输出，并在明确材料速度假设的基础上比较几何收缩和模型结构对烘干时间的影响。

二、问题分析

四问共享“附件数据重构—内部热湿传递—表面对流交换—数值求解—结果验证”的主链条，但各问新增困难不同。问题一解决基础的时变边界扩散；问题二处理系数随解变化引起的非线性；问题三引入全域阈值首达事件；问题四再处理随时间收缩的空间区域。按照这一依赖关系建立同一套守恒离散程序，可以避免各问采用不同数值口径造成的结果不可比。总体技术路线如图 1 所示。

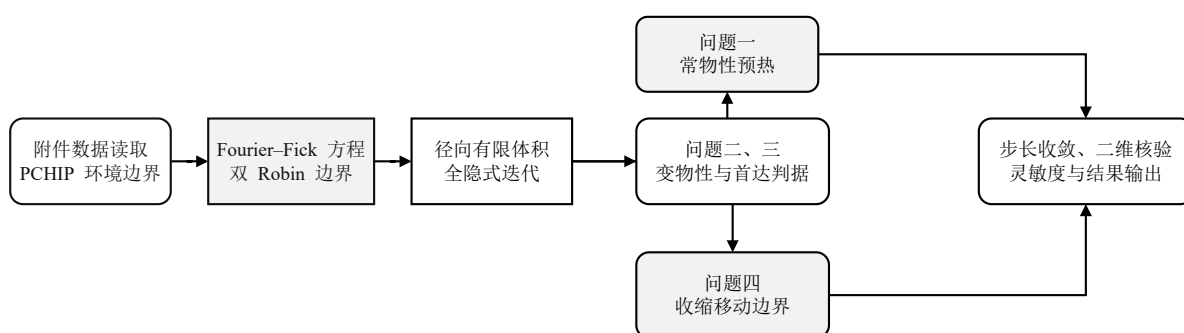


图 1 四问递进建模与验证技术路线

2.1 问题一的分析

问题一的物性和半径在计算时段内固定，因而温度方程是线性非稳态导热方程，水分方程仅因 $D(C)$ 保留弱非线性。环境温度和環境水分浓度每 60 s 给出一次，而结果要求每 1 s 输出，首先要在不引入振荡的前提下重构连续边界函数。若将药材近似为整体均匀温度和均匀含水率，会丢失题目明确要求的径向分布，也无法解释中心与表面响应的時間滞后。因此应采用圆柱坐标下的一维径向偏微分方程，并在轴线施加对称边界，在表面施加对流型 Robin 边界。

问题一同时承担数值基准的作用。其计算采用表面加密网格，再插值到题目要求的 0, 0.1, ..., 2.0 cm 输出位置；时间步取 1 s，与提交表时间分辨率一致。通过检查温度随半径递增、水分随半径递减、场量不越过环境和初值边界，可在进入变物性计算前确认边界符号及圆柱控制体面积处理正确。

2.2 问题二的分析

问题二给出的 $\rho(C)$, $c_p(C)$, $k(C)$ 和 $D(T, C)$ 使当前时间层的系数依赖未知温湿场。直接把旧时间层物性显式代入虽然计算简单，却会在长时间计算中引入明显的时间滞

后；完全联立 Newton 法则需要构造较复杂的块 Jacobian。考虑到两个方程的耦合主要通过物性传递，本文采用分步 Picard 迭代：先由当前迭代的水分更新热物性并求温度，再用新温度与当前水分更新扩散系数并求水分，直至两个场同时收敛。

此问还需要区分“温度接近环境”与“烘干完成”两个概念。热扩散率通常远大于有效水分扩散率，因此 3 h 时径向温差可能已经很小，而内部含水率仍高。分析时不仅报告各测点数值，还需比较中心—表面差及物性随干燥过程的变化，以说明后续时间为何由传质而非传热控制。

2.3 问题三的分析

“任意位置均不超过阈值”可写成空间最大值约束。由于表面直接与低水分环境交换，径向水分场预期保持中心高、表面低，但这一判断仍需由逐时间层的数值单调性检查确认。一旦中心被验证为全程最大点，原来的全空间终止条件便可转化为中心浓度的一维首达事件，从而可靠确定干燥结束时刻。

时间积分步长为 1 s，而提交结果每 60 s 输出，真实阈值时刻一般不落在任一输出点上。若直接取第一个达标输出时刻，会带来最多 60 s 的人为偏差。因此，应保存跨越阈值前后的两个内部时间层，依据控制浓度作线性事件插值，同时插值得到达标瞬间的完整剖面。随后再独立生成逐分钟结果表，使“物理首达精度”和“文件输出间隔”互不混淆。

2.4 问题四的分析

问题四的困难在于计算区域 $0 \leq r \leq R(t)$ 随时间收缩。若每一步在物理坐标上重新剖分网格，就必须进行场量重映射，容易造成额外的数值扩散和守恒误差。令 $\xi = r/R(t)$ 可把移动区域映射到固定区间 $[0, 1]$ ，材料速度与网格速度之差决定相对输运；在均匀材料收缩假设下两者相同，相对输运为零，而扩散系数前出现 R^{-2} 尺度因子、表面 Robin 项出现 R^{-1} 因子。

收缩模型的结果不能直接与问题三比较来判断“收缩贡献”，因为两问采用的物性函数不同。为隔离几何效应，需要另设一个使用附件四物性但保持 $R = 2 \text{ cm}$ 的反事实模型；再保留原固定空间标量描述作为结构对照，检查不同速度假设的影响，不将二者之差解释为独立的物理移动效应。固定物理位置超过当前半径时不再属于药材，结果表中必须留空，同时单列实时表面值。

三、模型假设

为使题目给出的数据能够封闭控制方程，并保证四问口径一致，作如下假设。

1. **连续介质与轴对称假设。**药材内部视为均匀、各向同性的连续多孔介质，初始温度和水分浓度均匀，烘房条件关于圆柱轴线对称，故不存在周向梯度。该假设使三维问题可先化为轴对称问题。
2. **中截面径向主导假设。**圆柱长径比为 6.25，题目只要求径向位置的输出，因此主模型描述远离端面的中截面径向场，暂不计轴向通量。端面影响不被直接忽略，而是在模型检验中用二维轴对称计算量化其适用范围。
3. **扩散主导假设。**药材内部热量由导热传递，水分由等效 Fick 扩散传递；忽略内部体热源、辐射、孔隙内宏观自然对流、重力渗流以及药材与托盘的接触传热。题目给出的等效物性被认为已经综合微观孔隙结构的平均影响。
4. **表面交换假设。**圆柱侧表面与环境通过对流换热和对流传质交换，系数分别取 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 与 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。附件三、四未给出新的表面系数，因此问题二至四沿用附件二数值，保证比较只由物性和几何变化引起。
5. **热湿耦合口径。**温度方程不额外加入汽化潜热项。题面未给出蒸发潜热、平衡吸湿曲线、相变界面和交叉输运系数，强行加入会产生无法由现有数据辨识的参数。本文以 $\rho(C)$, $c_p(C)$, $k(C)$ 和 $D(T, C)$ 表达题目明确给出的状态耦合，并在模型评价中说明该简化的影响。
6. **环境延拓假设。**附件一覆盖 $0 \sim 14400 \text{ s}$ 。问题三、四超过该时段后，环境温度和水分浓度分别保持附件末值 50.165°C 和 0.04986 kg/kg ，表示烘房进入稳定运行阶段；并通过 $\pm 5\%$ 扰动检验该延拓对达标时间的影响。
7. **均匀径向收缩假设。**问题四中长度保持 25 cm ，各材料点随半径同比例收缩，因而可用 $\xi = r/R(t)$ 标记材料坐标。干骨架质量守恒且其现体积密度在截面内均匀，满足 $\rho_d(t)R(t)^2 = \rho_d(0)R_0^2$ ；题给 ρc_p 用作有效储热系数，不另以它反演干骨架密度。附件二半径经单调 PCHIP 重构，不允许插值后出现局部回弹或超出数据范围。
8. **浓度量纲一致假设。**题目中的药材水分浓度和环境水分浓度均按 kg/kg 的同一干基比值处理，扩散方程与表面传质边界直接采用该量。若实际环境量为相对湿度，则需先通过吸湿等温线换算，本文不进行无数据支持的转换。

上述假设将未知的内部传热传质归结为可由题给物性和环境数据驱动的初边值问题。其中第 2、5、6 项对长期结果最敏感，故分别安排二维核验、模型局限讨论和边界灵敏度分析，而不是将其视为无需检验的先验事实。

四、符号说明

本文主要符号及其单位见表 1。温度函数中的绝对温度统一写为 $T_K = T + 273.15$ ，其余温差仍可直接用摄氏度计算。

表 1 主要符号及含义

符号	含义	单位	备注
r, z	径向、轴向物理坐标	m	主模型只保留 r
$R(t), L$	瞬时半径、圆柱长度	m	$L = 0.25$
T, T_e	药材温度、环境温度	°C	扩散系数中换为 K
C, C_e	药材、环境水分浓度	kg/kg	干基比值
ρ, c_p	密度、比热容	kg/m ³ , J/(kg K)	状态相关
k, D	导热系数、水分扩散系数	W/(m K), m ² /s	状态相关
h_T, h_m	表面传热、传质系数	W/(m ² K), m/s	Robin 边界
ξ	固定域材料坐标	1	$\xi = r/R(t)$
V_i, G_f	控制体体积、界面导通系数	m ² , 依场量而定	按单位轴向长度
$\Delta r_i, \Delta t$	局部空间、时间步长	m, s	表面加密网格, 1 s
n, m	时间层、Picard 迭代编号	1	非负整数
t_d	全截面达标首达时刻	s 或 h	$\max C \leq 0.15$

五、模型准备

5.1 几何降维与初始条件

在圆柱坐标 (r, θ, z) 中, 轴对称条件使 $\partial/\partial\theta = 0$ 。主模型进一步取中截面 $z = L/2$, 忽略该处的轴向梯度, 计算域为 $0 \leq r \leq R(t)$ 。固定半径问题中 $R = R_0 = 0.02$ m; 移动边界问题稍后映射到 $0 \leq \xi \leq 1$ 。初始条件为

$$T(r, 0) = 28 \text{ }^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}. \quad (1)$$

这一降维不是把整根药材等同为无空间差异的集中参数系统, 而是保留了题目要求的径向分布。其合理性来自圆柱较大的长径比, 并将在后文以包含端面的二维 r - z 模型进行独立检查。

5.2 附件数据检查与边界重构

附件数据的结构与采用方式汇总于表 2。附件一共有 241 个等间隔时刻, 附件二共有 145 个等间隔时刻, 二者采样频率与数值积分步长均不一致, 因此不能把原始测点直接当作逐步边界值。

对任一相邻数据点 (t_j, y_j) 、 (t_{j+1}, y_{j+1}) , 令 $\vartheta = (t - t_j)/(t_{j+1} - t_j)$, PCHIP 在区间内写为

$$y(t) = h_{00}(\vartheta)y_j + h_{10}(\vartheta)\Delta t_j d_j + h_{01}(\vartheta)y_{j+1} + h_{11}(\vartheta)\Delta t_j d_{j+1}, \quad (2)$$

表 2 附件时序数据及模型处理方式

数据	覆盖区间	采样间隔	模型处理
环境温度 T_e	0–14400 s	60 s	PCHIP, 区间外保持末值
环境水分 C_e	0–14400 s	60 s	PCHIP, 区间外保持末值
药材半径 R	0–259200 s	1800 s	单调 PCHIP 并同步求导

其中

$$h_{00} = 2\vartheta^3 - 3\vartheta^2 + 1, \quad h_{10} = \vartheta^3 - 2\vartheta^2 + \vartheta,$$

$$h_{01} = -2\vartheta^3 + 3\vartheta^2, \quad h_{11} = \vartheta^3 - \vartheta^2.$$

节点斜率 d_j 按相邻割线斜率的保形规则确定, 使单调数据区间不产生新的极值 [3]。相较普通三次样条, 该处理可避免半径在快速收缩段出现非物理回弹; 相较分段线性插值, 其一阶连续性又能稳定给出问题四所需的 $\dot{R}(t)$ 。

附件一的环境温度从 28 °C 上升至 50.165 °C, 环境水分浓度从 0.01963 变化至 0.04986 kg/kg。实测序列存在小幅波动, 故未强制其单调化; 只有半径数据按物理要求实施单调约束。超过 14400 s 后采用常值延拓, 从而不向长期计算人为加入新的升温程序。

5.3 径向热湿控制方程

基于 Fourier 导热定律和 Fick 扩散定律 [1], 固定半径域内的控制方程为

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right], \quad (3)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD(T, C)\frac{\partial C}{\partial r}\right]. \quad (4)$$

式中 ρc_p 是单位体积热容量, 决定温度对净热通量的响应速度; D 是以题设浓度变量定义的等效扩散系数。问题一中除 $D(C)$ 外各物性为常数, 问题二至四中所有系数均随状态更新。

轴线处没有净通量, 故满足

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0. \quad (5)$$

侧表面采用“向外法向通量等于表面对流通量”的约定：

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h_T [T(R, t) - T_e(t)], \quad (6)$$

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m [C(R, t) - C_e(t)]. \quad (7)$$

当环境温度高于表面温度时，(6) 给出指向药材内部的热流；当表面水分浓度高于环境时，(7) 给出向外水分通量。该符号检验可防止 Robin 条件写反后出现“越烘越湿”等非物理解。

5.4 物性关系及耦合路径

问题一使用附件二的常物性

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7.0 \times 10^{-9} \exp(-0.89/C). \quad (8)$$

问题二、三使用附件三关系

$$\begin{aligned} \rho &= 650 + 128C, & c_p &= 1450 + 2736 \frac{C}{C+1}, \\ k &= 0.21 + 0.38 \frac{C}{C+1}, & D &= 2.4 \times 10^{-3} \exp(-0.45/C) \exp[-3850/(T + 273.15)]. \end{aligned} \quad (9)$$

问题四使用附件四关系

$$\begin{aligned} \rho &= 760 + 90C, & c_p &= 1850 + 2150 \frac{C}{C+1}, \\ k &= 0.12 + 0.20 \frac{C}{C+1}, & D &= 4.2 \times 10^{-4} \exp(-0.30/C) \exp[-3850/(T + 273.15)]. \end{aligned} \quad (10)$$

由 (9) 和 (10) 可知，水分降低会同时改变储热能力和导热能力，温度升高则通过 Arrhenius 指数提高 D ；反过来，水分场的变化又会改变下一时间层的温度方程系数。本文所称热湿耦合即指这一“水分影响热物性、温度与水分共同影响扩散系数”的闭环，而非题面未给参数的汽化潜热耦合。

5.5 节点中心有限体积离散

空间节点取 $r_i = R[1 - (1 - i/N)^2]$, $i = 0, 1, \dots, N$, 基准取 $N = 160$, 对低扩散率表层加密；输出位置仍为每隔 0.1 cm, 由计算解插值得到。内部界面位于 $r_{i+1/2} = (r_i + r_{i+1})/2$ 。按单位轴向长度积分后，第 i 个环形控制体体积和界面面积分别为

$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}. \quad (11)$$

中心控制体下边界直接取 0，表面控制体上边界取 R 。这种几何构造自动消除了连续方程在 $r = 0$ 处形式上的 $1/r$ 奇点。

以 ϕ 统一表示 T 或 C ，储存系数记为 s ，扩散系数记为 Γ 。在时间层 $n + 1$ 采用全隐式格式，内部节点守恒式为

$$\frac{s_i V_i}{\Delta t} (\phi_i^{n+1} - \phi_i^n) = G_{i+1/2} (\phi_{i+1}^{n+1} - \phi_i^{n+1}) - G_{i-1/2} (\phi_i^{n+1} - \phi_{i-1}^{n+1}), \quad (12)$$

其中

$$G_{i+1/2} = \frac{A_{i+1/2} \Gamma_{i+1/2}}{r_{i+1} - r_i}, \quad \Gamma_{i+1/2} = \frac{\Gamma_i + \Gamma_{i+1}}{2}. \quad (13)$$

温度场取 $s = \rho c_p$, $\Gamma = k$ ，水分场取 $s = 1$, $\Gamma = D$ 。将 (12) 整理为

$$-a_W \phi_{i-1}^{n+1} + a_P \phi_i^{n+1} - a_E \phi_{i+1}^{n+1} = b_i, \quad (14)$$

其中

$$a_W = G_{i-1/2}, \quad a_E = G_{i+1/2}, \quad a_P = \frac{s_i V_i}{\Delta t} + a_W + a_E, \quad b_i = \frac{s_i V_i}{\Delta t} \phi_i^n.$$

矩阵主对角元为正，非对角元非正并满足对角占优，既保留局部守恒，也有利于保持温度、水分的离散最大值性质 [2]。

中心节点令 $a_W = 0$ ，自然实现零通量条件。表面节点将 Robin 通量直接并入控制体：

$$a_P \leftarrow a_P + A_R h, \quad b_N \leftarrow b_N + A_R h \phi_e, \quad A_R = 2\pi R, \quad (15)$$

其中热场取 $(h, \phi_e) = (h_T, T_e)$ ，水分场取 (h_m, C_e) 。最终每一场只需求解一个三对角线性系统，计算量随径向节点数线性增长。

5.6 非线性 Picard 迭代与输出控制

在每个新时间层，令初始迭代值等于上一时间层结果，并执行：

1. 由当前 $C^{(m)}$ 计算 ρ, c_p, k ，组装并求解温度三对角系统，得到 $T^{(m+1)}$ ；
2. 由 $T^{(m+1)}, C^{(m)}$ 计算 D ，组装并求解水分三对角系统，得到 $C^{(m+1)}$ ；
3. 检查

$$\|T^{(m+1)} - T^{(m)}\|_\infty < 10^{-8}, \quad \|C^{(m+1)} - C^{(m)}\|_\infty < 10^{-10};$$

若未同时满足则返回步骤 1，最多允许 30 次迭代。

四问终算均采用 $\Delta t = 1$ s；问题二、三共享从初态至达标的同一条轨迹，问题三按 60 s 抽取，不再独立积分。程序在每一步检查有限值、 $0 \leq C \leq 2.55$ 、径向单调性、Picard 收敛及方程平衡残差；阈值跨越时另存插值得到的首达剖面。空间网格、时间步长与输出采样分别检验，四位小数仅表示结果展示精度。

六、问题一：常物性预热过程模型的建立与求解

6.1 模型特化与特征尺度判断

问题一中半径固定为 $R_0 = 0.02 \text{ m}$ ，密度、比热容和导热系数分别为 820 kg/m^3 、 2600 J/(kgK) 和 0.36 W/(mK) 。因此热扩散率为

$$\alpha = \frac{k}{\rho c_p} = 1.689 \times 10^{-7} \text{ m}^2/\text{s}. \quad (16)$$

初始水分浓度下，由 (8) 得到

$$D_0 = 7.0 \times 10^{-9} \exp(-0.89/2.55) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}. \quad (17)$$

据此可估计径向热、质扩散特征时间

$$\tau_T = \frac{R_0^2}{\alpha} = 2369 \text{ s}, \quad \tau_C = \frac{R_0^2}{D_0} = 8.101 \times 10^4 \text{ s}. \quad (18)$$

两者相差约 34 倍，预示 1800 s 内温度扰动能够明显进入中心，而水分变化主要局限在外层。

进一步计算热、质 Biot 数

$$Bi_T = \frac{h_T R_0}{k} = 1.389, \quad Bi_C = \frac{h_m R_0}{D_0} = 3.240. \quad (19)$$

二者均明显大于集中参数法通常要求的 0.1，说明内部阻力不可忽略。尤其 $Bi_C > Bi_T$ ，表面对流传质并非唯一瓶颈，水分到达表面前必须克服更显著的内部扩散阻力。因此，本问不能只解两个平均状态常微分方程，而必须保留径向偏微分方程。

6.2 离散方案与求解步骤

取 $N = 160$ ，采用表面加密的 161 个计算节点，再线性插值到 $0, 0.1, \dots, 2.0 \text{ cm}$ 的 21 个输出位置。时间步长取 1 s ，从而每个内部时间层均可直接写入结果工作簿，不发生时间插值误差。温度方程代入常数 ρ, c_p, k 后只需在每个时间层更新环境边界；水分方程中的 $D(C)$ 仍随局部含水率变化，故与温度场一起按 Picard 框架求解。

具体计算顺序为：先由 PCHIP 读取 t_{n+1} 时刻的 T_e, C_e ；再组装温度三对角矩阵，求得 T^{n+1} ；随后依据当前水分迭代值更新各节点及界面的 D ，求得 C^{n+1} ；最后进行收敛、范围和单调性检查。问题一每个时间层最多 4 次 Picard 迭代即可达到给定容差。由于温度方程在本问中为线性，额外迭代主要用于水分扩散系数的自洽更新。

在物理边界上，中心节点不设置虚拟负半径点，而直接使用中心圆盘控制体和零内侧通量；表面节点使用半环形控制体，并将 $2\pi R_0 h_T$ 或 $2\pi R_0 h_m$ 加入矩阵末行。这种处

理同时保持圆柱几何和局部通量守恒，避免简单中心差分在 $r = 0$ 处分母为零的问题。

6.3 温度场计算结果与分析

问题一典型时刻的温度结果见表 3，完整径向曲线见图 2(a)。100 s 时热扰动尚集中在表面附近，中心温度只比初值高 $6.4 \times 10^{-5} \text{ }^{\circ}\text{C}$ ；随着热量持续输入，600 s 后整个截面开始明显升温。到 1800 s，中心与表面分别达到 $33.5770 \text{ }^{\circ}\text{C}$ 和 $36.7865 \text{ }^{\circ}\text{C}$ 。

表 3 问题一典型时刻温度分布 ($^{\circ}\text{C}$)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0004	28.0041	28.0324	28.1802
300	28.0414	28.0642	28.1522	28.3683	28.8495
600	28.4549	28.5377	28.8053	29.3168	30.1650
900	29.3262	29.4602	29.8774	30.6177	31.7305
1200	30.5447	30.7118	31.2241	32.1143	33.4304
1500	31.9976	32.1887	32.7681	33.7486	35.1218
1800	33.5770	33.7736	34.3654	35.3631	36.7865

温度剖面在各时刻均由中心向表面平滑升高，没有出现局部振荡或超过环境温度的节点。表面—中心温差由 100 s 的 $0.1801 \text{ }^{\circ}\text{C}$ 增至 1800 s 的 $3.2095 \text{ }^{\circ}\text{C}$ ，但温差增长速度逐渐降低：1500–1800 s 只增加 $0.0853 \text{ }^{\circ}\text{C}$ 。这表明表面首先响应烘房升温，随后热扩散不断向中心补偿，温度场正由强瞬态阶段向随环境缓慢变化的阶段过渡。

从 (18) 看，1800 s 已达到 $0.760\tau_T$ ，中心显著升温与尺度估计一致。剖面在靠近表面处斜率更大，对应 Robin 边界需要一定温差才能提供热通量；若错误地令表面温度等于环境温度，便会低估该外部换热阻力并高估内部升温速度。

6.4 水分场计算结果与分析

典型时刻水分浓度列于表 4，其空间曲线见图 2(b)。100 s 时表面水分已由 2.55 降至 2.2475 kg/kg ，而 $r \leq 1.0 \text{ cm}$ 区域在四位小数精度下仍保持初值。到 1800 s，表面进一步降至 1.5104 kg/kg ，中心仍为 2.5500 kg/kg ，形成 1.0396 kg/kg 的中心—表面差。

温度和水分的中心—表面差随时间变化汇总于表 5。温差和浓度差均单调扩大，但二者成因不同：温差扩大来自环境持续升温与内部热惯性的共同作用；水分差扩大则来自很小的 Fourier 质扩散数。1800 s 时

$$Fo_C = \frac{D_0 t}{R_0^2} = 0.0222,$$

表 4 问题一典型时刻水分浓度 (kg/kg)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8772
900	2.5500	2.5500	2.5497	2.5045	1.7549
1200	2.5500	2.5500	2.5482	2.4646	1.6587
1500	2.5500	2.5499	2.5445	2.4206	1.5789
1800	2.5500	2.5497	2.5383	2.3755	1.5104

远小于 1，因而内部水分来不及充分重新分布。

表 5 问题一中心—表面差的演化

t/s	$T(R, t) - T(0, t)/^{\circ}\text{C}$	$C(0, t) - C(R, t)/(\text{kg/kg})$
100	0.1801	0.3025
300	0.8081	0.4989
600	1.7101	0.6728
900	2.4043	0.7951
1200	2.8857	0.8913
1500	3.1242	0.9711
1800	3.2095	1.0396

此外，(8) 显示 C 降低会使指数 $-0.89/C$ 更负，即干燥后的表层扩散系数反而减小。由此形成的“低扩散率干层”进一步阻碍内部水分外移，使剖面在 $r > 1.5 \text{ cm}$ 处明显变陡。这一机制解释了表面下降很快而中心几乎不变，并说明不能用表面水分直接代表截面平均或中心水分。

6.5 本问结论与完整结果说明

问题一建立了四问共享的圆柱径向守恒求解框架。数值结果同时满足温度“内低外高”、水分“内高外低”的物理方向，各时刻剖面光滑，中心对称条件和表面 Robin 通量均得到正确体现。预热 1800 s 尚未使内部水分发生显著变化，因而烘干控制对象应是中心水分而非表面水分。

本问内部结果逐秒保存至 1800 s，并插值到 21 个固定径向位置；正文表格和图形均读取同一终算数据。四位小数为展示格式，不代表所有末位均达到同等数值精度。

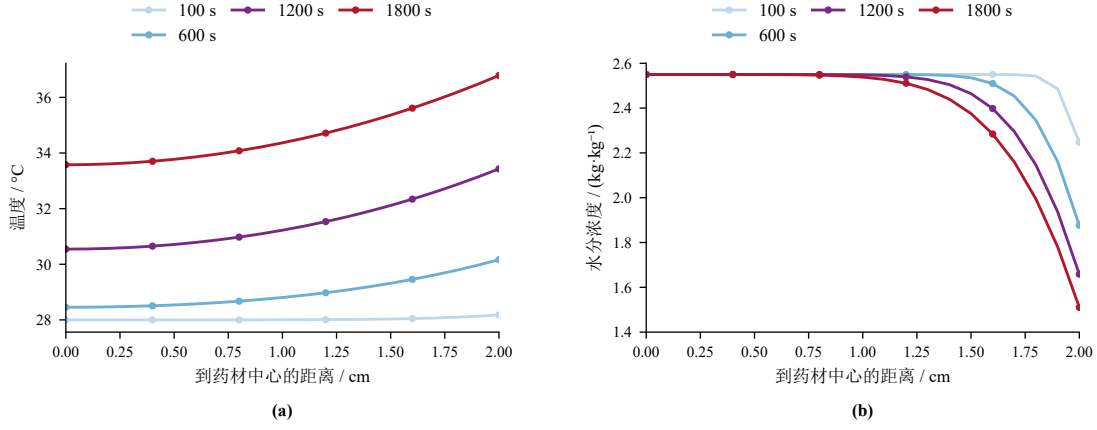


图2 问题一温度与水分径向剖面

七、问题二：变物性热湿迁移模型的建立与求解

7.1 非线性模型及耦合关系

问题二保持 $R = 0.02 \text{ m}$ 、 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 和 $h_m = 8 \times 10^{-7} \text{ m/s}$ 不变，将 (9) 代入 (3) 和 (4)。此时温度方程的储存项和扩散项都依赖 C ，水分方程的扩散系数同时依赖 T 和 C 。若写成算子形式，可表示为

$$\mathcal{H}[T; C] = 0, \quad \mathcal{M}[C; T] = 0, \quad (20)$$

其中两个算子通过状态相关系数双向联系。该耦合不要求在一个大矩阵中同时求解全部未知量：在每次 Picard 迭代内，先固定 C 解 $\mathcal{H}$ ，再用更新后的 T 解 $\mathcal{M}$ ，即可逐步逼近同一时间层的自洽状态。

与问题一相比，问题二的初始状态物性并不等于附件二常数。将 $T = 28 \text{ }^\circ\text{C}$ 、 $C = 2.55 \text{ kg/kg}$ 代入附件三关系，得到初始密度 976.4 kg/m^3 、比热容 $3415.3 \text{ J}/(\text{kgK})$ 、导热系数 $0.4830 \text{ W}/(\text{mK})$ 和扩散系数 $5.642 \times 10^{-9} \text{ m}^2/\text{s}$ 。因此，两问在相同几何和表面对流系数下仍会呈现不同响应，不能把问题一的 1800 s 结果直接作为问题二的初段结果。

7.2 物性随状态的演化

为展示非线性系数的实际变化，将初态以及 3 h 的中心、表面状态代入 (9)，结果见表 6。随着水分降低， ρ, c_p, k 均减小；这意味着单位体积储热能力降低，同时内部导热能力也有所下降。二者对升温速度具有相反影响，需要由完整能量方程共同决定。

扩散系数同时受温度和水分影响。升温使 Arrhenius 因子 $\exp[-3850/(T + 273.15)]$ 增大，促进水分扩散；干燥使 $\exp(-0.45/C)$ 减小，抑制扩散。3 h 时两种效应叠加后，中心和表面 D 都高于初值，但表面尽管温度略高，其 D 反而小于中心值。这说明低含水表层已开始形成扩散阻力，水分迁移并非随温度升高而无限加速。

表 6 问题二代表状态下的物性参数

状态	$\rho/(\text{kg/m}^3)$	$c_p/[\text{J}/(\text{kg K})]$	$k/[\text{W}/(\text{m K})]$	$D/(\text{m}^2/\text{s})$
初始全域	976.4	3415.3	0.4830	5.642×10^{-9}
3 h 中心	876.1	3196.9	0.4526	1.239×10^{-8}
3 h 表面	779.0	2823.5	0.4008	1.027×10^{-8}

7.3 迭代求解与收敛表现

问题二使用 161 个表面加密径向节点和 1 s 时间步，从 0 连续积分至烘干达标后的首个整分钟；正文保留前 3 h 的代表结果。对第 $n+1$ 层，令

$$T^{(0)} = T^n, \quad C^{(0)} = C^n,$$

每次迭代先计算 $\rho(C^{(m)})$, $c_p(C^{(m)})$, $k(C^{(m)})$ 并求 $T^{(m+1)}$ ，再计算 $D(T^{(m+1)}, C^{(m)})$ 并求 $C^{(m+1)}$ 。全程时间层均在最多 5 次迭代内达到双重无穷范数容差，没有触及 30 次迭代上限。说明在本题状态范围和 1 s 步长下，分步 Picard 映射具有良好的局部收敛性。

全隐式格式对扩散问题不存在显式格式的严格稳定步长限制，但时间步仍决定截断误差和边界序列解析度。选取 1 s 不仅与题目输出一致，也远小于附件一 60 s 的采样间隔，可充分解析 PCHIP 重构后的环境变化。三对角系统规模为 161，故逐秒输出并不会造成不可接受的计算负担。

7.4 温度结果及其空间演化

典型时刻温度见表 7，空间曲线见图 3(a)。0.5 h 时中心和表面分别为 32.1911 °C、35.4141 °C，仍有 3.2230 °C 的明显差异；1.5 h 后差异降为 1.2928 °C；到 3 h 仅剩 0.1173 °C。这表明温度先经历表面快速升温 and 内部滞后，随后在环境逐渐稳定时趋于径向均匀。

表 7 问题二温度分布 (°C)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.1911	32.3840	32.9674	33.9620	35.4141
1	40.3822	40.5543	41.0609	41.8784	43.0001
1.5000	45.8464	45.9345	46.1927	46.6049	47.1393
2	48.4498	48.4877	48.5980	48.7732	49.0030
2.5000	49.4667	49.4790	49.5133	49.5645	49.6617
3	49.8493	49.8552	49.8746	49.9100	49.9666

在同为 1800 s 的时刻，问题一中心与表面温度为 33.5770 °C 和 36.7865 °C，问题二则为 32.1911 °C 和 35.4141 °C。问题二初始体积热容量 ρc_p 较大，单位温升需要吸收更多热量，因此早期温度低于问题一；随着水分下降，体积热容量逐渐减小，后续升温速度提高，3 h 时全截面已接近烘房稳定温度。

7.5 水分结果及控制机制

水分结果见表 8 和图 3(b)。0.5 h 时中心仍为 2.5499 kg/kg，表面已降至 1.6487 kg/kg；3 h 时中心和表面分别降至 1.7662 和 1.0081 kg/kg。中心—表面差由 0.5 h 的 0.9012 降至 3 h 的 0.7581 kg/kg，说明水分扰动已经传播到中心，但内部尚未达到近似均匀状态。

表 8 问题二水分浓度分布 (kg/kg)

<i>t</i> /h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5489	2.5255	2.3257	1.6487
1	2.5257	2.4947	2.3578	2.0230	1.4712
1.5000	2.3860	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6260	1.2311
2.5000	1.9567	1.9006	1.7360	1.4720	1.1167
3	1.7662	1.7165	1.5702	1.3333	1.0081

与温度不同，水分曲线在 3 h 时仍保持明显曲率。其原因一是水分扩散特征时间较长，二是表层含水下降会降低局部 D ，形成动态内部阻力。虽然表面水分浓度已经低于初值的 40%，中心仍保留约 69% 的初始水分。由此可判断：当烘房温度和药材温度看似已经稳定时，烘干过程仍远未结束，终止判据必须施加在内部最大水分浓度上。

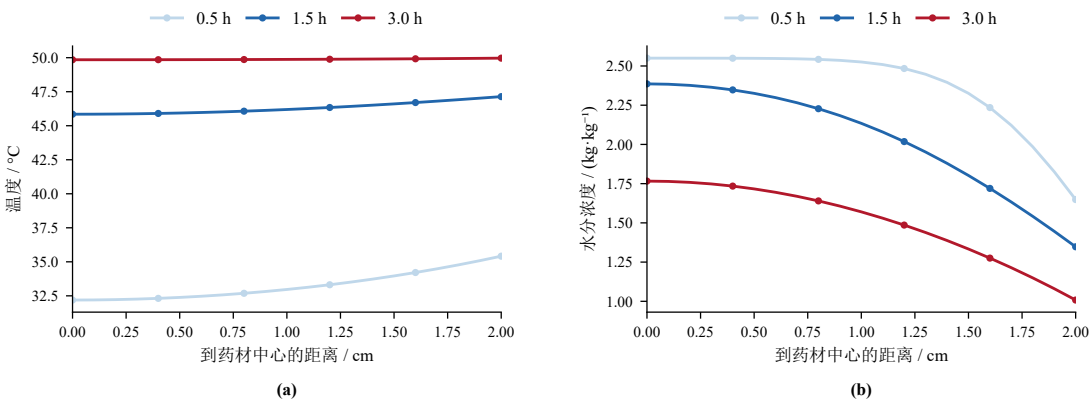


图 3 问题二变物性温度与水分剖面

7.6 本问结论与结果文件

问题二完成了由常物性基准模型向状态耦合非线性模型的扩展。结果揭示：温度场约在 3 h 内接近径向均匀，而水分场仍有 0.7581 kg/kg 的中心—表面差；在此之后，环境热量主要通过提高 $D(T, C)$ 促进内部水分迁移，而非继续显著改变温度分布。

本问内部结果完整保留至烘干结束，每秒保存温度与水分场；问题三的逐分钟轨迹直接从该序列抽取，相同时间、位置的温湿数值及首达时刻完全一致。

八、 问题三：全截面达标时间模型的建立与求解

8.1 全域首达判据

问题三要求“所有位置”的水分浓度均不超过 0.15 kg/kg，因此不能用截面平均值或表面值代替。定义空间最大水分浓度

$$M(t) = \max_{0 \leq r \leq R_0} C(r, t), \quad (21)$$

则烘干完成时刻是集合约束的首次满足时间

$$t_d = \inf\{t \geq 0 : M(t) \leq 0.15\}. \quad (22)$$

该定义包含两个必要条件：一是 t_d 时全空间均达标，二是在任何 $t < t_d$ 时至少存在一个未达标位置。因而它比“取第一个保存时刻”更严格，也能避免输出间隔改变时答案随之改变。

初始水分场均匀，表面环境水分始终显著低于药材内部，且有限体积矩阵是对角占优的 M 型结构。在这种向外失水的条件下，数值解应保持由中心向表面非增。程序对每一个内部时间层检查

$$C_i^n - C_{i+1}^n \geq -2 \times 10^{-9}, \quad i = 0, 1, \dots, N-1, \quad (23)$$

全部时间层均通过。因此

$$M(t) = C(0, t), \quad (24)$$

即轴线是本问的控制位置。这里并非预先假定中心最大，而是先保留全域最大值判据，再由计算结果确认其可化为中心事件。

8.2 阈值事件定位方法

长期积分使用 $\Delta t = 1 \text{ s}$ 。设第 n 层尚未达标而第 $n + 1$ 层已经达标，即

$$M^n > 0.15, \quad M^{n+1} \leq 0.15.$$

在这两个相邻时间层间定义

$$\lambda = \frac{0.15 - M^n}{M^{n+1} - M^n}, \quad t_d = t_n + \lambda \Delta t, \quad (25)$$

并同步插值得到达标剖面

$$C(r, t_d) = C^n(r) + \lambda[C^{n+1}(r) - C^n(r)]. \quad (26)$$

由于 $0 \leq \lambda \leq 1$ ，所得时刻位于真实跨越发生的 1 s 小区间内。首达事件采用内部步长定位，而结果三工作簿仍按题意每 60 s 输出；两种时间分辨率彼此独立。

计算时从 $t = 0$ 直接连续推进，并在 14400 s 后保持附件一末端环境。每个时间层继续使用问题二的附件三物性和 Picard 迭代，不在 3 h 处重启或重新初始化，问题三的逐分钟结果由问题二完整逐秒序列抽取，共用一次终算。

8.3 干燥时间与径向分布

由 (25) 得到

$$t_d = 205808.3592 \text{ s} = 57.1690 \text{ h}. \quad (27)$$

典型时刻及达标瞬间的水分浓度见表 9。中心从 6 h 的 1.0156 降至 12 h 的 0.4548 ，随后下降速度逐渐减慢；达标时中心恰为 0.1500 ，而表面已降至 0.0525 kg/kg ，接近稳定环境水分浓度。

如图 4(a) 所示，中心和表面水分均先快速下降、后逐渐变缓。表面在 12.5414 h 已先达到 0.15 ，而中心直到 57.1690 h 才达标，两者相差 44.6276 h 。若烘干设备只依据表面传感器停机，会在中心仍明显超标时提前终止，这是本题采用全域判据而非表面判据的直接工程意义。

8.4 干燥阶段与速率变化

以每 6 h 的中心水分差估计分段平均下降速率， $6\text{--}12 \text{ h}$ 为 $0.09347 (\text{kg/kg})/\text{h}$ ， $12\text{--}18 \text{ h}$ 降至 0.02615 ， $24\text{--}30 \text{ h}$ 为 0.00531 ， $48\text{--}54 \text{ h}$ 仅为 0.00132 。后者只有前者的约 $1/71$ 。这种显著的降速并非环境温度继续下降导致： 4 h 后环境已保持稳定，主要原因是水分浓度降低使 D 中的 $\exp(-0.45/C)$ 项快速减小，同时中心到表面的有效扩散路径仍保持 2 cm 。

表 9 问题三每 6 h 及达标时的水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0156	0.9868	0.9006	0.7547	0.5335
12	0.4548	0.4418	0.4020	0.3289	0.1640
18	0.2979	0.2906	0.2679	0.2247	0.0842
24	0.2371	0.2321	0.2161	0.1851	0.0663
30	0.2053	0.2013	0.1887	0.1637	0.0597
36	0.1854	0.1821	0.1714	0.1500	0.0565
42	0.1717	0.1687	0.1593	0.1404	0.0547
48	0.1615	0.1588	0.1504	0.1332	0.0536
54	0.1535	0.1511	0.1433	0.1275	0.0528
57.1690	0.1500	0.1477	0.1402	0.1249	0.0525

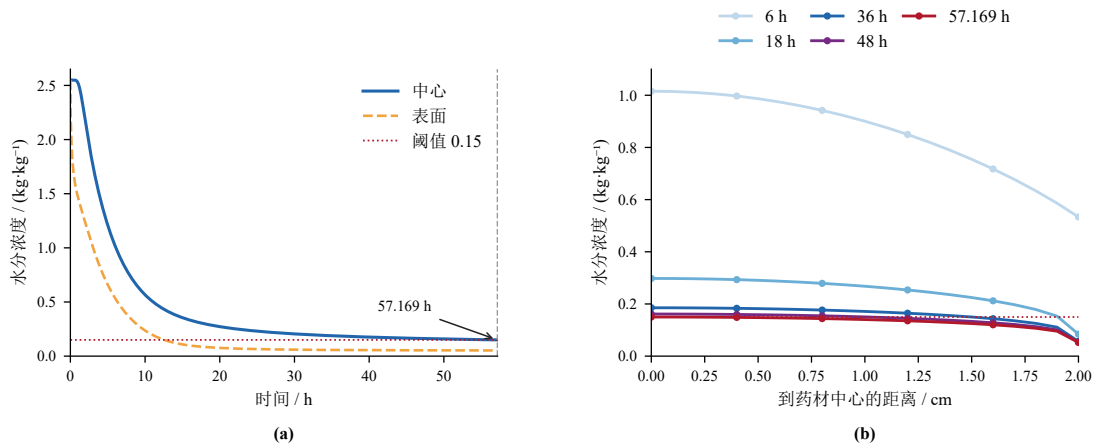


图 4 问题三阈值首达过程与径向分布

图 4(b) 进一步显示, 早期剖面整体较高且表面梯度大; 进入后期后, 外层很快贴近环境水平, 剩余水分主要集中在中心附近。48 h 至达标时, 各径向位置仍保持严格的内高外低次序, 但中心附近曲线较平, 意味着最终阶段需要等待深部水分穿过已经较干的外层。这也解释了中心水分从 0.1615 降至 0.1500 仍需要约 9.17 h。

8.5 精度与结果文件说明

采用 2 s 步长重复求解得到 57.1699 h, 与 1 s 终算的相对差为 1.52×10^{-5} , 绝对差约 3.14 s。161 与 321 个加密节点的空间检验相对差为 1.66×10^{-5} (统一取 10 s)。这些差值独立于 60 s 的保存间隔; 达标剖面满足中心等于阈值、其余节点不超过阈值, 所有长期剖面通过 (23)。

本问内部逐分钟序列保存至 205860 s，覆盖真实首达时刻 205808.3592 s；21 个半径列对应固定物理位置。正文达标剖面采用 (26) 的事件值，与最后一个整分钟样本明确区分。

九、问题四：收缩移动边界模型的建立与求解

9.1 半径数据重构与固定域映射

附件二给出的半径由初始 2 cm 持续减小至 1.2 cm。经单调 PCHIP 重构后，在本问主要时刻的半径见表 10。前 6 h 半径已经缩小到 1.374 cm，完成总收缩量的 78.3%；此后收缩速度逐渐减小，36 h 后在四位小数精度下稳定为 1.200 cm。由此可见，几何变化主要集中在干燥早期，但它对随后全部扩散过程都会持续产生影响。

表 10 问题四典型时刻的重构半径

t/h	0	6	12	18	24	30	36	48	50.821
R/cm	2.000	1.374	1.248	1.214	1.204	1.201	1.200	1.200	1.200

若直接在物理域 $0 \leq r \leq R(t)$ 上使用固定的 0.1 cm 网格，节点数会随时间减少，旧节点场量还需频繁插值到新网格，不利于保持守恒。为此定义无量纲固定域坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (28)$$

并令 $\psi(\xi, t) = \phi(r, t)$ 。半径收缩时，固定 ξ 的节点随材料截面同步向内移动，始终保留 161 个表面加密计算节点，不需要改变矩阵维数。

9.2 移动边界方程推导

为区分材料运动与网格运动，记固体速度为 v_s ，网格速度为 $w = \xi \dot{R}$ 。链式法则给出材料导数

$$\frac{\partial \phi}{\partial t} + v_s \frac{\partial \phi}{\partial r} = \left. \frac{\partial \psi}{\partial t} \right|_{\xi} + \frac{v_s - w}{R} \frac{\partial \psi}{\partial \xi}. \quad (29)$$

本问采用均匀材料收缩闭合： $v_s = w$ ，干骨架密度 ρ_d 在截面内均匀且满足 $\rho_d(t)R^2(t) = \rho_{d0}R_0^2$ 。以干基水分 C 表示水质量，取相对扩散通量 $j_w = -\rho_d D \partial_r C$ ，则干骨架连续性与水质量平衡相减后给出 $\rho_d(\partial_t C + v_s \partial_r C) = r^{-1} \partial_r(r \rho_d D \partial_r C)$ 。因此材料坐标方程为

$$\frac{\partial \psi}{\partial t} = \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi \Gamma \frac{\partial \psi}{\partial \xi} \right), \quad (30)$$

水分取 $\psi = C, \Gamma = D$; 温度取 $\psi = T, \Gamma = k$, 并在左侧乘以题给有效储热系数 ρc_p 。该热方程沿用题给物性的有效描述, 不把 $\rho(C)/(1+C)$ 同时作为独立干骨架密度, 否则需额外检验两者是否兼容。收缩干燥的物质平衡须明确内部速度及边界通量 [6]; 本文均匀闭合是一项需实验验证的假设, 不能仅由坐标变换证明。

在此闭合下, 相对输运速度为

$$u_{\text{rel}} = \frac{v_s - w}{R} = 0. \quad (31)$$

因此无需另加迎风项。收缩主要通过 R^{-2} 的扩散尺度及表面交换尺度进入方程; 只把固定空间扩散方程作坐标变换而保留 $-\dot{R}\xi/R$, 对应另一种速度描述, 不等价于随材料运动的干基水分平衡。

表面边界也必须同步变换。由 $\partial/\partial r = R^{-1}\partial/\partial \xi$ 得

$$-\frac{k}{R} \frac{\partial T}{\partial \xi} \Big|_{\xi=1} = h_T(T_s - T_e), \quad (32)$$

$$-\frac{D}{R} \frac{\partial C}{\partial \xi} \Big|_{\xi=1} = h_m(C_s - C_e). \quad (33)$$

因此在无量纲控制体方程中, 内部扩散导通系数乘 R^{-2} , 表面对流导通系数乘 R^{-1} 。若只缩放网格而遗漏这些尺度因子, 模型将不能正确反映半径变化带来的通量放大。

9.3 材料坐标离散与迭代流程

在 $\xi_i = 1 - (1 - i/160)^2$ 上采用控制体界面通量离散, 径向内部导通系数取 R^{-2} 尺度, 表面项取 R^{-1} 尺度。均匀收缩下干质量对应固定的参考体积权重 V_i , 全域水分平衡可逐步检验为

$$\sum_i V_i (C_i^{m+1} - C_i^m) + \Delta t \frac{2\pi h_m}{R^{n+1}} (C_s^{m+1} - C_e^{m+1}) = 0, \quad (34)$$

其中 $V_i = \pi(\xi_{i+1/2}^2 - \xi_{i-1/2}^2)$, 省略共同的干质量因子, 内部界面通量两两抵消。第四问累计归一化质量平衡残差为 1.77×10^{-13} , 有效热方程的最大单步归一化残差为 3.38×10^{-14} ; 后者不代表包含蒸发潜热的总焓守恒。

每个时间层先由 PCHIP 更新半径、环境量及附件四物性, 再依次求解温度与水分三对角系统。Picard 最大迭代次数为 5, 平均为 4.15。完整逻辑见图 5: 内层判断控制物性迭代, 外层判断全域阈值; 未收敛时继续当前时间层迭代, 未达标时推进至下一时间层。

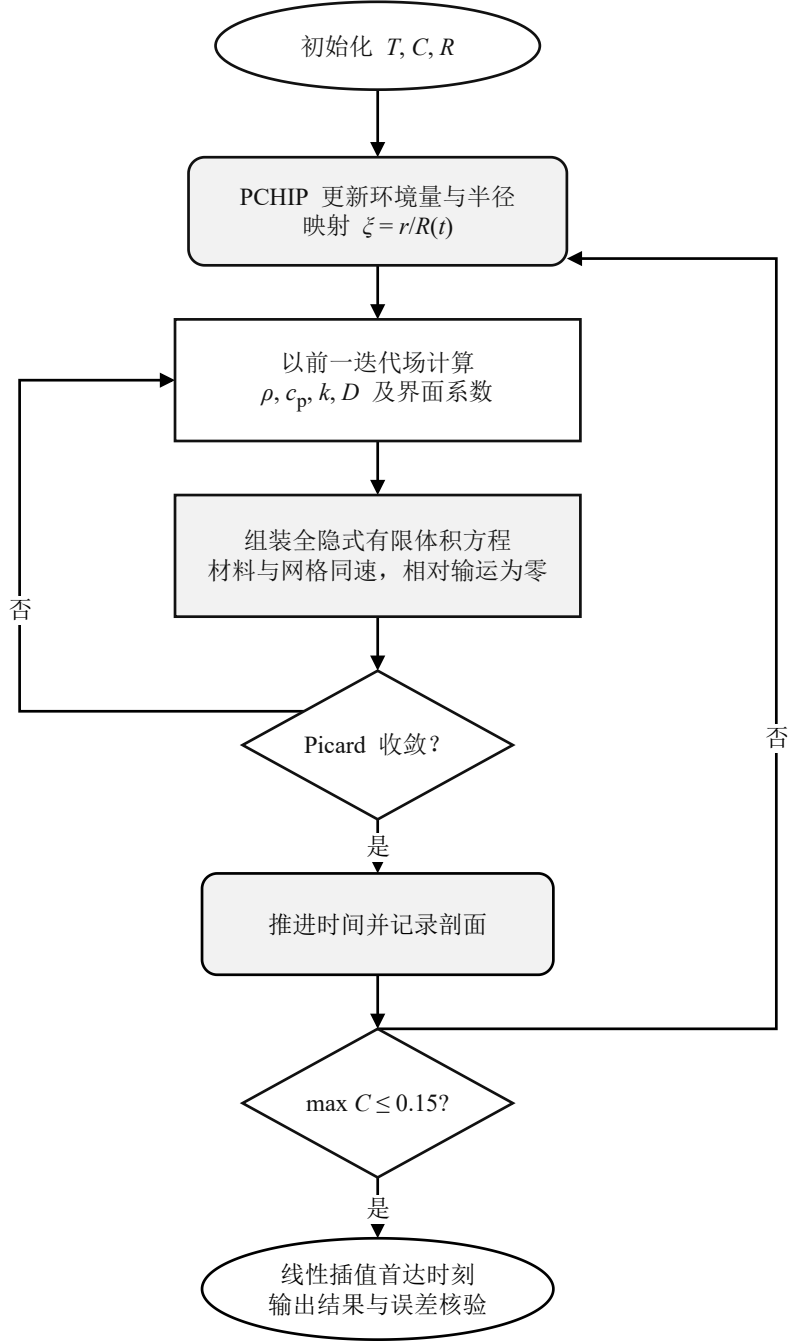


图 5 问题四移动边界全隐式迭代流程

9.4 物理位置输出规则

计算解首先定义在固定材料坐标 ξ 上, 而题目模板给出的列对应固定物理位置 $r_j = 0, 0.1, \dots, 1.9 \text{ cm}$ 。对任一时刻, 仅当 $r_j \leq R(t)$ 时计算

$$C(r_j, t) = \mathcal{I}_\xi[C(\xi, t)] \Big|_{\xi=r_j/R(t)}, \quad (35)$$

其中 $\mathcal{I}_\xi$ 为固定域节点间线性插值; 当 $r_j > R(t)$ 时, 该位置已经处于药材外部, 单元格留空而不是填 0。最后一列始终记录 $C(\xi = 1, t)$, 即随半径移动的实时表面。

这一规则区分了“固定空间传感器位置”和“随材料移动的表面位置”。例如 6 h 时 $R = 1.374 \text{ cm}$ ，故 $r = 1.4 \text{ cm}$ 及更外列均应留空；而 $r = 1.0 \text{ cm}$ 仍位于药材内部，其对应材料坐标已由初始的 0.5 变化为约 0.728。若继续把 $r = 2.0 \text{ cm}$ 列当作表面，就会把药材外部的环境量误认为材料状态。

9.5 达标时间与浓度分布

问题四继续以全域最大水分浓度作为终止量。全部剖面保持中心最大，故事件定位仍可用 (25) 和 (26)。计算得到

$$t_d = 182956.8045 \text{ s} = 50.8213 \text{ h}, \quad R(t_d) = 1.2000 \text{ cm}. \quad (36)$$

典型时刻的固定物理位置与动态表面水分见表 11。6 h 时中心水分仍为 1.7164，动态表面已降至 0.4215 kg/kg；24 h 时中心为 0.2837，表面只有 0.0671；达标时中心为 0.1500，表面为 0.0525。

表 11 问题四典型时刻水分浓度 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	动态表面
6	1.7164	1.5343	1.0206	0.4215
12	0.7341	0.6516	0.4067	0.1666
18	0.4065	0.3666	0.2383	0.0888
24	0.2837	0.2597	0.1780	0.0671
30	0.2253	0.2084	0.1485	0.0593
36	0.1920	0.1789	0.1310	0.0558
42	0.1706	0.1598	0.1194	0.0539
48	0.1556	0.1463	0.1111	0.0529
50.8213	0.1500	0.1412	0.1079	0.0525

动态表面在 12.7685 h 已达到 0.15，而中心需要 50.8213 h，两者仍相差 38.0528 h。收缩虽然显著缩短了内部扩散距离，却没有消除“外干内湿”的过程特征，因此实际控制仍应关注中心或内部代理量。

9.6 收缩效应的反事实分解

问题三和问题四的物性函数不同，直接比较 57.1690 h 与 50.8213 h 不能将全部差异归因于收缩。为保持物性和环境相同，另作两组对照：

1. 使用附件四物性和相同环境，但保持 $R = 2 \text{ cm}$ ，得到固定半径反事实时长 129.0934 h；

2. 保留真实 $R(t)$ ，采用原固定空间标量方程的速度描述，得到 52.3656 h，仅用作模型结构比较。

第一组在本文材料收缩闭合下给出

$$\Delta t_R = 129.0934 - 50.8213 = 78.2720 \text{ h}, \quad \eta_R = \frac{\Delta t_R}{129.0934} = 60.63\%. \quad (37)$$

第二组与材料模型相差 1.5442 h，表明速度描述影响长期预测。该差异不能解释为独立“坐标移动效应”，也不能据此认定任一方程已被实验验证。固定半径与均匀收缩模型的中心水分轨迹及半径变化见图 6。

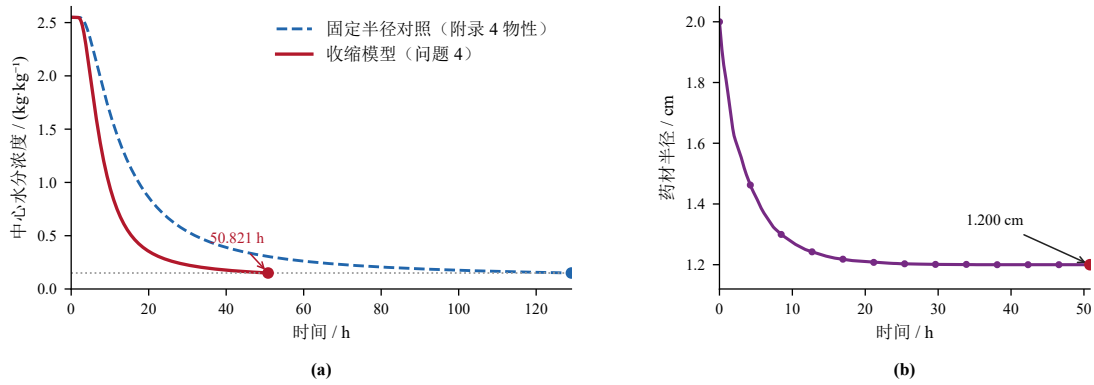


图 6 问题四收缩与固定半径反事实比较

9.7 数值检查与结果文件

时间步由 2 s 减为 1 s，达标时间由 50.8220 h 变为 50.8213 h，相对差为 1.30×10^{-5} ；161 与 321 个加密节点的空间检验相对差为 6.09×10^{-7} （统一取 10 s）。半径单调非增，水分剖面中心最大，全部 Picard 迭代收敛，且通过参考干质量与有效热方程平衡检查。这些检验支持所列假设下的数值一致性，不代替对真实收缩机制的实验辨识。

本问内部结果每分钟保存至 183000 s，覆盖真实首达时刻。固定物理位置与动态表面的转换遵循 (35)，超出材料区域的空值表示几何域外，不应填零。

十、模型的检验、评价与推广

10.1 空间网格与时间步长检验

全隐式稳定性不能代替精度检验。计算网格采用 $r_i/R = 1 - (1 - i/N)^2$ ，将输出位置与计算节点分开。先统一取 10 s 检查 41、81、161、321 个节点，再固定 161 个节点将步长由 2 s 减为 1 s，结果见表 12。161 至 321 节点的达标时间相对变化分别为 1.66×10^{-5} 、

6.09×10^{-7} ; 2 s 至 1 s 的变化分别为 1.52×10^{-5} 、 1.30×10^{-5} , 均小于预设空间 10^{-3} 、时间 10^{-4} 的相对容差。

表 12 网格与步长收敛检验：达标时间 (h)

计算设置	问题三	问题四
41 节点, 10 s	57.158718	50.826704
81 节点, 10 s	57.173087	50.827137
161 节点, 10 s	57.176830	50.827258
321 节点, 10 s	57.177777	50.827289
161 节点, 2 s	57.169860	50.821993
161 节点, 1 s (终算)	57.168989	50.821335

上述比较给出网格与步长变化的经验量级, 并非严格误差上界。阈值在内部相邻时间层间定位, 60 s 保存间隔不决定积分精度; 四位小数仅为统一展示格式。问题二、三共享终算轨迹, 公共时刻温湿场逐元素完全一致。四问累计归一化水分方程残差最大为 6.88×10^{-13} , 有效热方程单步归一化残差最大为 3.38×10^{-14} ; 后者只检验所实现方程, 未包含蒸发潜热和完整总焓。

10.2 二维轴对称独立核验

另建立 $0 \leq r \leq R(t)$, $0 \leq z \leq L/2$ 的半长圆柱轴对称有限体积模型, 本节将 $z = 0$ 定为中截面, $z = L/2$ 为端面。轴线和中截面零通量, 侧表面与端面施加相同热、质 Robin 边界。问题一在 41 个径向、21 个半轴向加密节点及 10 s 步长下核验至 1800 s; 问题三、四延伸至全二维区域烘干完成, 采用 30 s 步长, 并与相同径向网格和步长的一维计算配对比较。

问题一全时段中截面最大温差为 $7.49 \times 10^{-7} \text{ }^{\circ}\text{C}$, 最大水分差为 $7.35 \times 10^{-14} \text{ kg/kg}$ 。长期 41×21 网格下, 问题三、四的二维达标时间分别为 57.173517、50.839856 h, 与配对一维计算的相对差分别为 4.59×10^{-5} 、 1.87×10^{-7} ; 两问全二维最大水分均位于轴线与中截面交点。全过程中截面最大温差分别为 0.001775、0.000108 $^{\circ}\text{C}$, 最大水分差分别为 2.41×10^{-5} 、 $1.38 \times 10^{-6} \text{ kg/kg}$ 。

进一步加密至 81×41 节点后, 两问二维达标时间分别为 57.188295、50.840290 h, 与配对一维的相对差分别为 3.87×10^{-5} 、 1.59×10^{-7} , 控制位置未变。这项加密比较支持端部影响结论; 二维检验的 30 s 步长与 1 s 终算不同, 不直接用二维时长替换主模型答案。

这些量为所有内部时间层上的最大值, 与仅比较末时刻不同。端面附近仍明显较干, 长期最大端面-中截面水分差分别约为 0.970 和 1.707 kg/kg。因此二维检验支持当前均

匀各向同性、给定端面交换与收缩假设下的中心控制；一维剖面仍只代表中部，不能直接解释端面局部状态，也不构成对其他端部条件的普遍保证。

10.3 环境边界灵敏度分析

长期结果依赖 4 h 后的环境延拓。温度采用明确的加性扰动 $T_{e,\infty} \pm \delta T$ ，其中 $\delta T = 2.50825^\circ\text{C}$ ；环境水分采用 $C_{e,\infty}(1 \pm 0.05)$ 。前 4 h 边界不变，所有基准与扰动计算统一使用 161 个节点及 10 s 步长。第四问的半径历史保持给定曲线不变，因此这里只检验边界条件的条件灵敏度，不包含环境改变引起的收缩反馈。结果见表 13，表中基准与 1 s 终算略有差异。

表 13 环境参数扰动下的干燥时长 (h)

情形	问题三	问题四
基准 (10 s)	57.1768	50.8273
末端温度 -2.50825°C	61.9880	55.0444
末端温度 $+2.50825^\circ\text{C}$	52.8757	47.0517
末端环境水分 -5%	57.1293	50.7754
末端环境水分 $+5\%$	57.2290	50.8861

为避免摄氏温标零点改变无量纲百分比，温度先报告有量纲对称差分导数，水分报告无量纲弹性：

$$K_T = \frac{t_d(T_e + \delta T) - t_d(T_e - \delta T)}{2\delta T}, \quad S_C = \frac{t_d(1.05C_e) - t_d(0.95C_e)}{0.10t_d(C_e)}. \quad (38)$$

问题三、四的 K_T 分别为 -1.8165 、 $-1.5933 \text{ h}^\circ\text{C}$ ， S_C 分别为 0.0174 、 0.0218 。若需无量纲温度弹性，应以绝对温度定义 $S_T = (T_{e,K}/t_d)K_T$ ，两问分别为 -10.2715 、 -10.1349 ，而不是将摄氏数值直接作百分比归一化。

温度升高缩短干燥时间，环境水分升高延长干燥时间。在本组扰动幅度下，温度影响明显大于环境水分；这一方向与 $D(T, C)$ 中 Arrhenius 温度因子一致。该比较针对给定扰动、延拓时段和固定半径历史，不是包含几何反馈的总灵敏度。

10.4 模型优点

本文模型具有以下特点：

1. **统一性。**四问均由同一 Fourier–Fick 控制方程和有限体积离散出发，新增物性、阈值与收缩时只扩展相应模块，避免结果口径分裂。

2. **守恒性。**有限体积法直接对环形控制体平衡通量，中心奇点和表面 Robin 边界都有明确几何意义；三对角矩阵结构也便于检查系数符号和对角占优。
3. **可解释性。**特征时间解释问题一的热快湿慢，状态物性解释问题二的干层阻力，首达事件解释问题三的中心控制，反事实模型则把问题四的几何作用与物性差异分开。
4. **可复现性。**环境和半径均采用确定性的 PCHIP，数值程序无随机过程；论文表格与数据图来自同一终算文件，并通过网格、步长、方程平衡和长期二维多重检查。
5. **输出完整性。**固定域解与固定物理位置输出被明确区分，落到收缩药材外部的列按几何规则留空，真实首达时间也不被 60 s 输出间隔量化。

10.5 模型局限

首先，热湿耦合只通过题给物性函数体现，没有显式描述水分蒸发消耗的潜热。若真实蒸发冷却显著，本文可能高估材料升温速度并进一步高估 $D(T, C)$ 。但在缺少潜热、平衡吸湿关系和相变边界数据时，加入该项需要额外参数，不能由现有附件唯一确定。

其次，主模型代表中截面而非端面。长期二维核验在当前端面条件下确认轴线与中截面交点控制达标；若长度、端部交换条件或各向异性改变，仍需重新核验全域控制点。再次，材料被视为均匀各向同性；实际药材的纤维方向、表皮和内部空腔可能使径向与轴向扩散系数不同。

最后，4 h 后环境采用常值延拓，虽然灵敏度分析表明结论对小幅扰动稳定，但它仍是长期预测的主要外部不确定性来源。问题四又假设收缩为均匀材料径向缩放，并区分守恒干骨架密度与题给有效热物性；这些闭合尚未经实验辨识，也没有考虑局部开裂、翘曲和长度变化。本文结果因此更适合作为给定附件条件下的理论预测和控制基线，而非未经标定即可替代实际烘干试验。

10.6 工程应用与模型推广

从控制角度看，表面温度和表面水分都过早接近环境或阈值，不能单独作为停机信号。可在少量试验中布置中心温湿传感器，用本文模型反演或校正 D, h_m ，随后建立由环境历史和表面观测预测中心水分的降阶代理模型。控制器以预测中心水分首次达到 0.15 为停机条件，可兼顾内部达标与表层不过度干燥。

根据问题一至三的阶段特征，还可构造两阶段温度策略：早期在允许的品质温度范围内提高环境温度，利用较高 D 加快内部扩散；后期中心接近阈值时降低温度或采用间歇均湿，使内外水分重新分配。具体最优温度轨迹需要增加品质损失、能耗和设备上下限等目标与约束，本文框架可直接作为最优控制中的状态方程。

若获得蒸发潜热、平衡含湿曲线和交叉输运系数，可将 (3) 和 (4) 推广为 Luikov 型热湿耦合方程 [4]；若药材长度较短或端面换热强化，则采用本文二维核验模型；若收缩

出现各向异性, 则把单一 $R(t)$ 扩展为轴向、径向两个尺度函数。因而, 该方法不仅适用于中药材, 也可用于果蔬、食品、木材及其他多孔生物材料的变物性干燥和收缩预测。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具, 主要用于题意整理、候选建模思路比较、代码调试和语言润色, 详细使用情况见支撑材料。题目附件均由程序直接读取, 控制方程、边界条件、数值算法、结果文件和图表均由本地代码独立计算, 并通过时间步长减半、二维轴对称模型、物理单调性和文件回读进行人工复核。参赛队对论文中的模型选择、计算结果和文字表述承担最终责任。

参考文献

- [1] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [2] Patankar S V. Numerical Heat Transfer and Fluid Flow[M]. Washington, DC: Hemisphere, 1980.
- [3] Fritsch F N, Carlson R E. Monotone piecewise cubic interpolation[J]. SIAM Journal on Numerical Analysis, 1980, 17(2): 238–246. DOI: 10.1137/0717021.
- [4] Luikov A V. Systems of differential equations of heat and mass transfer in capillary-porous bodies[J]. International Journal of Heat and Mass Transfer, 1975, 18(1): 1–14.
- [5] Adrover A, Brasiello A. A moving boundary model for food isothermal drying and shrinkage[J]. Journal of Food Process Engineering, 2019, 42(7): e13178. DOI: 10.1111/jfpe.13178.
- [6] Adrover A, Brasiello A. The role of shrinkage on food isothermal drying: a moving boundary model[J]. Chemical Engineering Transactions, 2019, 74: 1111–1116. DOI: 10.3303/CET1974186.

附录 A 支撑材料文件列表

表 14 支撑材料及作用

文件	作用
<code>solve_drying.py</code>	四问有限体积求解、首达判定、移动边界与步长检验主程序
<code>write_results.py</code>	四份提交工作簿的生成、格式处理和回读核验
<code>validate_2d.py</code>	二维轴对称端面效应独立核验
<code>run_sensitivity.py</code>	环境温度及水分浓度的灵敏度分析
<code>make_figures.py</code>	论文科研图的可复现绘制与矢量导出
<code>result1.xlsx</code>	问题一每 1 s、0.1 cm 的温度和水分结果
<code>result2.xlsx</code>	问题二每 1 s、0.1 cm 的温度和水分结果
<code>result3.xlsx</code>	问题三每 60 s、0.1 cm 的水分结果
<code>result4.xlsx</code>	问题四每 60 s、固定位置及动态表面结果
<code>drawio/*.drawio</code>	总体技术路线与移动边界求解流程图源文件
<code>AI_USAGE.md</code>	AI 工具、使用环节、用途及人工核验记录
<code>VERIFY_REPORT.md</code>	数值、工作簿、图形和论文最终验收记录

复现时先运行 `solve_drying.py` 生成四问终算场，再运行工作簿、二维核验、灵敏度和绘图脚本。全部程序使用固定网格与确定性线性代数，不含随机过程；论文附录 B 嵌入的主程序与实际运行文件一致。

附录 B 完整源程序

以下为实际运行的四问核心求解程序全文，按源文件顺序单栏连续排版；电子支撑材料中另提供原始 Python 文件。

```
"""Numerical solution for CUMCM 2026 Problem A.

The model uses a node-centred finite-volume discretisation in the radial
coordinate, fully implicit diffusion, Picard iteration for nonlinear
properties, and an ALE-style fixed-domain transform for shrinkage.
"""

from __future__ import annotations

import argparse
import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator
from scipy.linalg import solve_banded

ROOT = Path(__file__).resolve().parents[1]
INPUT_DIR = ROOT / "附件"
RESULTS_DIR = ROOT / "results"
H_T = 25.0
H_M = 8.0e-7
R_FIXED = 0.02
NODES = 161
OUTPUT_NODES = 21
GRID_POWER = 2.0
INITIAL_T = 28.0
INITIAL_C = 2.55
THRESHOLD_C = 0.15

@dataclass
class Environment:
    time_s: np.ndarray
    temperature_c: np.ndarray
    moisture: np.ndarray
```

```

temp_interp: PchipInterpolator
moisture_interp: PchipInterpolator

def values(
    self,
    time_s: float,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
) -> tuple[float, float]:
    cutoff = float(self.time_s[-1])
    if time_s <= cutoff:
        return float(self.temp_interp(time_s)), float(self.moisture_interp(time_s))
    return (
        float(self.temperature_c[-1] * terminal_temp_scale),
        float(self.moisture[-1] * terminal_moisture_scale),
    )

@dataclass
class RadiusHistory:
    time_s: np.ndarray
    radius_m: np.ndarray
    interp: PchipInterpolator
    derivative: PchipInterpolator
    monotone_adjustments: int

    def values(self, time_s: float) -> tuple[float, float]:
        bounded = min(max(time_s, float(self.time_s[0])), float(self.time_s[-1]))
        return float(self.interp(bounded)), float(self.derivative(bounded))

def _read_numeric_sheet(path: Path) -> np.ndarray:
    book = load_workbook(path, read_only=True, data_only=True)
    sheet = book.active
    rows: list[list[float]] = []
    for row in sheet.iter_rows(min_row=2, values_only=True):
        if row[0] is None:
            continue
        rows.append([float(value) for value in row])
    return np.asarray(rows, dtype=float)

def load_environment() -> Environment:
    data = _read_numeric_sheet(INPUT_DIR / "附件1.xlsx")
    time_s, temperature_c, moisture = data.T
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件1时间列不是严格递增。")

```

```

if not np.all(np.isfinite(data)):
    raise ValueError("附件1含非有限数。")
return Environment(
    time_s=time_s,
    temperature_c=temperature_c,
    moisture=moisture,
    temp_interp=PchipInterpolator(time_s, temperature_c),
    moisture_interp=PchipInterpolator(time_s, moisture),
)

def load_radius_history() -> RadiusHistory:
    data = _read_numeric_sheet(INPUT_DIR / "附件2.xlsx")
    time_s = data[:, 0]
    raw_radius_m = data[:, 1] / 100.0
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件2时间列不是严格递增。")
    monotone_radius_m = np.minimum.accumulate(raw_radius_m)
    adjustments = int(np.count_nonzero(np.abs(monotone_radius_m - raw_radius_m) > 1e-12))
    interp = PchipInterpolator(time_s, monotone_radius_m, extrapolate=False)
    return RadiusHistory(
        time_s=time_s,
        radius_m=monotone_radius_m,
        interp=interp,
        derivative=interp.derivative(),
        monotone_adjustments=adjustments,
    )

def property_q1(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    del temperature_c
    rho = np.full_like(moisture, 820.0)
    cp = np.full_like(moisture, 2600.0)
    k = np.full_like(moisture, 0.36)
    safe_c = np.maximum(moisture, 1e-8)
    diffusivity = 7.0e-9 * np.exp(-0.89 / safe_c)
    return rho, cp, k, diffusivity

def property_q23(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 650.0 + 128.0 * safe_c
    cp = 1450.0 + 2736.0 * safe_c / (safe_c + 1.0)
    k = 0.21 + 0.38 * safe_c / (safe_c + 1.0)
    diffusivity = (
        2.4e-3
        * np.exp(-0.45 / safe_c)
    )

```

```

        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def property_q4(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 760.0 + 90.0 * safe_c
    cp = 1850.0 + 2150.0 * safe_c / (safe_c + 1.0)
    k = 0.12 + 0.20 * safe_c / (safe_c + 1.0)
    diffusivity = (
        4.2e-4
        * np.exp(-0.30 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def face_interpolate(left: np.ndarray, right: np.ndarray) -> np.ndarray:
    """Linear face interpolation for a smooth state-dependent coefficient."""
    return 0.5 * (left + right)

def radial_grid(count: int, radius: float = 1.0) -> np.ndarray:
    """Surface-refined computational mesh; output positions are independent."""
    s = np.linspace(0.0, 1.0, count)
    return radius * (1.0 - (1.0 - s) ** GRID_POWER)

def node_control_volumes(coordinate: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return dimensionless/physical radial CV volumes and internal face radii."""
    faces = 0.5 * (coordinate[:-1] + coordinate[1:])
    lower = np.concatenate(([0.0], faces))
    upper = np.concatenate((faces, [coordinate[-1]]))
    volumes = math.pi * (upper**2 - lower**2)
    return volumes, faces

def implicit_radial_step(
    old: np.ndarray,
    storage: np.ndarray,
    conductivity: np.ndarray,
    dt: float,
    coordinate: np.ndarray,
    diffusion_scale: float,
    boundary_conductance: float,
    boundary_value: float,

```

```

    advection_velocity: np.ndarray | None = None,
) -> np.ndarray:
    """One implicit node-centred radial FVM step on coordinate [0, 1] or [0, R]."""
    count = len(old)
    volumes, faces = node_control_volumes(coordinate)
    delta = np.diff(coordinate)
    face_property = face_interpolate(conductivity[:-1], conductivity[1:])
    conductance = 2.0 * math.pi * faces * face_property * diffusion_scale / delta

    capacity = storage * volumes / dt
    diagonal = capacity.copy()
    lower = np.zeros(count - 1)
    upper = np.zeros(count - 1)
    rhs = capacity * old

    diagonal[:-1] += conductance
    diagonal[1:] += conductance
    upper -= conductance
    lower -= conductance

    diagonal[-1] += boundary_conductance
    rhs[-1] += boundary_conductance * boundary_value

    if advection_velocity is not None:
        # PDE form:  $\phi_t + u(\xi) \phi_{\xi} = \text{diffusion}$ ;  $u \geq 0$  under shrinkage.
        advective = storage[1:] * volumes[1:] * np.maximum(advection_velocity[1:], 0.0) / delta
        diagonal[1:] += advective
        lower -= advective

    matrix = np.zeros((3, count))
    matrix[0, 1:] = upper
    matrix[1, :] = diagonal
    matrix[2, :-1] = lower
    return solve_banded((1, 1), matrix, rhs, check_finite=False)

def advance_coupled(
    temperature_old: np.ndarray,
    moisture_old: np.ndarray,
    dt: float,
    radius_m: float,
    radius_rate_m_s: float,
    ambient_temperature_c: float,
    ambient_moisture: float,
    property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
    moving_domain: bool,
    tolerance_temperature: float = 1e-8,

```

```

tolerance_moisture: float = 1e-10,
max_iterations: int = 30,
balance: dict | None = None,
) -> tuple[np.ndarray, np.ndarray, int]:
    temperature = temperature_old.copy()
    moisture = moisture_old.copy()
    if moving_domain:
        coordinate = radial_grid(len(temperature_old))
        diffusion_scale = 1.0 / radius_m**2
        boundary_scale = 1.0 / radius_m
        shrink_velocity = -(radius_rate_m_s / radius_m) * coordinate
    else:
        coordinate = radial_grid(len(temperature_old), radius_m)
        diffusion_scale = 1.0
        boundary_scale = 1.0
        shrink_velocity = None

    for iteration in range(1, max_iterations + 1):
        rho, cp, k, _ = property_model(temperature, moisture)
        heat_storage = rho * cp
        heat_boundary = 2.0 * math.pi * coordinate[-1] * H_T * boundary_scale
        next_temperature = implicit_radial_step(
            old=temperature_old,
            storage=heat_storage,
            conductivity=k,
            dt=dt,
            coordinate=coordinate,
            diffusion_scale=diffusion_scale,
            boundary_conductance=heat_boundary,
            boundary_value=ambient_temperature_c,
            advection_velocity=shrink_velocity,
        )

        _, _, _, diffusivity = property_model(next_temperature, moisture)
        mass_boundary = 2.0 * math.pi * coordinate[-1] * H_M * boundary_scale
        next_moisture = implicit_radial_step(
            old=moisture_old,
            storage=np.ones_like(moisture),
            conductivity=diffusivity,
            dt=dt,
            coordinate=coordinate,
            diffusion_scale=diffusion_scale,
            boundary_conductance=mass_boundary,
            boundary_value=ambient_moisture,
            advection_velocity=shrink_velocity,
        )

```

```

        error_t = float(np.max(np.abs(next_temperature - temperature)))
        error_c = float(np.max(np.abs(next_moisture - moisture)))
        temperature = next_temperature
        moisture = next_moisture
        if error_t < tolerance_temperature and error_c < tolerance_moisture:
            break
    else:
        raise RuntimeError(
            f"Picard iteration failed: dT={error_t:.3e}, dC={error_c:.3e}, dt={dt}"
        )

if not np.all(np.isfinite(temperature)) or not np.all(np.isfinite(moisture)):
    raise FloatingPointError("Non-finite state generated.")
if np.min(moisture) < -1e-10 or np.max(moisture) > INITIAL_C + 1e-8:
    raise FloatingPointError("Moisture left physical bounds.")
if balance is not None:
    volumes, _ = node_control_volumes(coordinate)
    adv_c = adv_t = 0.0
    if shrink_velocity is not None:
        weight = volumes[1:] * np.maximum(shrink_velocity[1:], 0.0) / np.diff(coordinate)
        adv_c = dt * float(np.dot(weight, np.diff(moisture)))
        adv_t = dt * float(np.dot(weight * heat_storage[1:], np.diff(temperature)))
    change = float(np.dot(volumes, moisture - moisture_old))
    flux = dt * mass_boundary * (float(moisture[-1]) - ambient_moisture)
    reference = INITIAL_C * float(volumes.sum())
    balance['moisture_change'] += change / reference
    balance['boundary_loss'] += flux / reference
    balance['relative_transport'] += adv_c / reference
    residual = change + flux + adv_c
    balance['maximum_step_mass_residual'] = max(balance['maximum_step_mass_residual'],
        abs(residual) / reference)
    heat_change = float(np.dot(heat_storage * volumes, temperature - temperature_old))
    heat_flux = dt * heat_boundary * (float(temperature[-1]) - ambient_temperature_c)
    heat_reference = INITIAL_T * float(np.dot(heat_storage, volumes))
    heat_residual = (heat_change + heat_flux + adv_t) / heat_reference
    balance['maximum_step_heat_equation_residual'] =
        max(balance['maximum_step_heat_equation_residual'], abs(heat_residual))
    return temperature, moisture, iteration

def new_balance() -> dict:
    return dict(moisture_change=0.0, boundary_loss=0.0, relative_transport=0.0,
        maximum_step_mass_residual=0.0, maximum_step_heat_equation_residual=0.0)

def simulate_fixed(
    environment: Environment,

```

```

property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
end_time_s: float | None,
dt_s: float,
output_interval_s: float,
stop_at_threshold: bool,
terminal_temp_scale: float = 1.0,
terminal_moisture_scale: float = 1.0,
stop_interval_s: float | None = None,
) -> dict[str, np.ndarray | float | int | bool]:
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    radial_m = radial_grid(NODES, R_FIXED)
    output_r = np.linspace(0.0, R_FIXED, OUTPUT_NODES)
    times = [0.0]
    temperatures = [np.interp(output_r, radial_m, temperature)]
    moistures = [np.interp(output_r, radial_m, moisture)]
    iterations: list[int] = []
    monotone_profiles = True
    center_is_max = True
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    stop_output_time = math.inf
    balance = new_balance()
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(end_time_s if end_time_s is not None else 10.0 * 24.0 * 3600.0)

    while time_s < maximum_time - 1e-9:
        step = min(dt_s, maximum_time - time_s, next_output - time_s)
        new_time = time_s + step
        temperature_before = temperature.copy()
        moisture_before = moisture.copy()
        ambient_t, ambient_c = environment.values(
            new_time, terminal_temp_scale, terminal_moisture_scale
        )
        temperature, moisture, count = advance_coupled(
            temperature,
            moisture,
            step,
            R_FIXED,
            0.0,
            ambient_t,
            ambient_c,
            property_model,

```

```

        moving_domain=False,
        balance=balance,
    )
    iterations.append(count)
    time_s = new_time
    current_max = float(np.max(moisture))
    monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
    center_is_max = center_is_max and bool(
        moisture[0] >= float(np.max(moisture)) - 1e-9
    )

    if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
        fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e-15)
        threshold_time = previous_time + fraction * step
        threshold_temperature = (
            temperature_before + fraction * (temperature - temperature_before)
        )
        threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
        stop_spacing = output_interval_s if stop_interval_s is None else stop_interval_s
        stop_output_time = math.ceil(threshold_time / stop_spacing) * stop_spacing

    if time_s + 1e-8 >= next_output:
        times.append(time_s)
        temperatures.append(np.interp(output_r, radial_m, temperature))
        moistures.append(np.interp(output_r, radial_m, moisture))
        next_output += output_interval_s

    if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
        break

    previous_max = current_max
    previous_time = time_s

    return {
        "times_s": np.asarray(times),
        "radius_m": output_r,
        "temperature_c": np.asarray(temperatures),
        "moisture": np.asarray(moistures),
        "threshold_time_s": threshold_time,
        "threshold_temperature_c": None if threshold_temperature is None else
np.interp(output_r, radial_m, threshold_temperature),
        "threshold_moisture": None if threshold_moisture is None else np.interp(output_r,
radial_m, threshold_moisture),
        "last_time_s": time_s,
        "picard_max": int(max(iterations, default=0)),
        "picard_mean": float(np.mean(iterations) if iterations else 0.0),
        "monotone_profiles": monotone_profiles,
        "center_is_max": center_is_max,
    }

```

```

        "balance": balance,
    }

def simulate_moving(
    environment: Environment,
    radius_history: RadiusHistory,
    dt_s: float,
    output_interval_s: float,
    stop_at_threshold: bool = True,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
    include_advection: bool = False,
) -> dict[str, np.ndarray | float | int | bool]:
    # Default: homogeneous solid shrinkage, material velocity equals mesh velocity.
    # True retains the former stationary-spatial scalar model for structural comparison.
    xi = radial_grid(NODES)
    output_xi = np.linspace(0.0, 1.0, OUTPUT_NODES)
    temperature = np.full(NODES, INITIAL_T)
    moisture = np.full(NODES, INITIAL_C)
    times = [0.0]
    radii = [R_FIXED]
    temperatures = [np.interp(output_xi, xi, temperature)]
    moistures = [np.interp(output_xi, xi, moisture)]
    iterations: list[int] = []
    previous_max = float(np.max(moisture))
    previous_time = 0.0
    threshold_time = math.nan
    threshold_temperature: np.ndarray | None = None
    threshold_moisture: np.ndarray | None = None
    threshold_radius_m = math.nan
    stop_output_time = math.inf
    time_s = 0.0
    next_output = output_interval_s
    maximum_time = float(radius_history.time_s[-1])
    monotone_profiles = True
    center_is_max = True
    maximum_cfl = 0.0
    balance = new_balance()

    while time_s < maximum_time - 1e-9:
        step = min(dt_s, maximum_time - time_s, next_output - time_s)
        new_time = time_s + step
        temperature_before = temperature.copy()
        moisture_before = moisture.copy()
        radius_m, radius_rate_m_s = radius_history.values(new_time)
        if not include_advection:

```

```

        radius_rate_m_s = 0.0
    ambient_t, ambient_c = environment.values(
        new_time, terminal_temp_scale, terminal_moisture_scale
    )
    temperature, moisture, count = advance_coupled(
        temperature,
        moisture,
        step,
        radius_m,
        radius_rate_m_s,
        ambient_t,
        ambient_c,
        property_q4,
        moving_domain=True,
        balance=balance,
    )
    iterations.append(count)
    time_s = new_time
    current_max = float(np.max(moisture))
    maximum_cfl = max(
        maximum_cfl,
        float(np.max(abs(radius_rate_m_s / radius_m) * xi[1:] * step / np.diff(xi))),
    )
    monotone_profiles = monotone_profiles and bool(np.all(np.diff(moisture) <= 2e-9))
    center_is_max = center_is_max and bool(
        moisture[0] >= float(np.max(moisture)) - 1e-9
    )

    if previous_max > THRESHOLD_C >= current_max and math.isnan(threshold_time):
        fraction = (previous_max - THRESHOLD_C) / max(previous_max - current_max, 1e-15)
        threshold_time = previous_time + fraction * step
        threshold_temperature = (
            temperature_before + fraction * (temperature - temperature_before)
        )
        threshold_moisture = moisture_before + fraction * (moisture - moisture_before)
        threshold_radius_m = radius_history.values(threshold_time)[0]
        stop_output_time = (
            math.ceil(threshold_time / output_interval_s) * output_interval_s
        )

    if time_s + 1e-8 >= next_output:
        times.append(time_s)
        radii.append(radius_m)
        temperatures.append(np.interp(output_xi, xi, temperature))
        moistures.append(np.interp(output_xi, xi, moisture))
        next_output += output_interval_s

```

```

        if stop_at_threshold and time_s + 1e-8 >= stop_output_time:
            break
        previous_max = current_max
        previous_time = time_s

    return {
        "times_s": np.asarray(times),
        "xi": output_xi,
        "radii_m": np.asarray(radii),
        "temperature_c": np.asarray(temperatures),
        "moisture": np.asarray(moistures),
        "threshold_time_s": threshold_time,
        "threshold_temperature_c": None if threshold_temperature is None else
np.interp(output_xi, xi, threshold_temperature),
        "threshold_moisture": None if threshold_moisture is None else np.interp(output_xi, xi,
threshold_moisture),
        "threshold_radius_m": threshold_radius_m,
        "last_time_s": time_s,
        "picard_max": int(max(iterations, default=0)),
        "picard_mean": float(np.mean(iterations) if iterations else 0.0),
        "monotone_profiles": monotone_profiles,
        "center_is_max": center_is_max,
        "maximum_advection_cfl": maximum_cfl,
        "balance": balance,
        "velocity_model": "stationary_spatial_comparison" if include_advection else
"uniform_material_shrinkage",
    }

def extract_fixed_table(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_times_s: list[float],
    requested_radii_cm: list[float],
    variable: str,
) -> list[list[float]]:
    times = np.asarray(simulation["times_s"])
    radii_m = np.asarray(simulation["radius_m"])
    values = np.asarray(simulation[variable])
    table: list[list[float]] = []
    for requested_time in requested_times_s:
        threshold_time = float(simulation.get("threshold_time_s", math.nan))
        threshold_key = (
            "threshold_temperature_c" if variable == "temperature_c" else "threshold_moisture"
        )
        threshold_profile = simulation.get(threshold_key)
        if (
            threshold_profile is not None

```

```

        and math.isfinite(threshold_time)
        and abs(requested_time - threshold_time) < 1e-5
    ):
        profile = np.asarray(threshold_profile)
    else:
        time_index = int(np.argmin(np.abs(times - requested_time)))
        profile = values[time_index]
    row = [
        float(np.interp(radius_cm / 100.0, radii_m, profile))
        for radius_cm in requested_radii_cm
    ]
    table.append(row)
return table

def extract_moving_profile(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_time_s: float,
    requested_radii_cm: list[float],
) -> list[float | None]:
    times = np.asarray(simulation["times_s"])
    xi = np.asarray(simulation["xi"])
    radii_m = np.asarray(simulation["radii_m"])
    moisture = np.asarray(simulation["moisture"])
    threshold_time = float(simulation.get("threshold_time_s", math.nan))
    threshold_profile = simulation.get("threshold_moisture")
    if (
        threshold_profile is not None
        and math.isfinite(threshold_time)
        and abs(requested_time_s - threshold_time) < 1e-5
    ):
        profile = np.asarray(threshold_profile)
        radius = float(simulation["threshold_radius_m"])
    else:
        index = int(np.argmin(np.abs(times - requested_time_s)))
        profile = moisture[index]
        radius = float(radii_m[index])
    values: list[float | None] = []
    for radius_cm in requested_radii_cm:
        position_m = radius_cm / 100.0
        if position_m > radius + 1e-12:
            values.append(None)
        else:
            values.append(float(np.interp(position_m / radius, xi, profile)))
    values.append(float(profile[-1]))
    return values

```

```

def save_npz(path: Path, simulation: dict[str, np.ndarray | float | int | bool]) -> None:
    arrays = {key: value for key, value in simulation.items() if isinstance(value, np.ndarray)}
    np.savez_compressed(path, **arrays)

def json_ready(value):
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, float) and math.isnan(value):
        return None
    if isinstance(value, dict):
        return {key: json_ready(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [json_ready(item) for item in value]
    return value

def write_summary(path: Path, payload: dict) -> None:
    path.write_text(
        json.dumps(json_ready(payload), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def run_all(run_convergence: bool = True) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    for name in ("q1", "q2", "q3", "q4"):
        (RESULTS_DIR / name).mkdir(parents=True, exist_ok=True)

    environment = load_environment()
    radius_history = load_radius_history()
    data_audit = {
        "environment_rows": len(environment.time_s),
        "environment_time_range_s": [environment.time_s[0], environment.time_s[-1]],
        "environment_sampling_s": np.unique(np.diff(environment.time_s)),
        "temperature_range_c": [
            float(np.min(environment.temperature_c)),
            float(np.max(environment.temperature_c)),
        ],
        "ambient_moisture_range": [
            float(np.min(environment.moisture)),
            float(np.max(environment.moisture)),
        ],
        "radius_rows": len(radius_history.time_s),
    }

```

```

    "radius_time_range_s": [radius_history.time_s[0], radius_history.time_s[-1]],
    "radius_range_cm": [
        float(np.min(radius_history.radius_m) * 100.0),
        float(np.max(radius_history.radius_m) * 100.0),
    ],
    "radius_monotone_adjustments": radius_history.monotone_adjustments,
}
write_summary(RESULTS_DIR / "data_audit.json", data_audit)

q1 = simulate_fixed(
    environment,
    property_q1,
    end_time_s=1800.0,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=False,
)
save_npz(RESULTS_DIR / "q1" / "fields.npz", q1)
q1_times = [100, 300, 600, 900, 1200, 1500, 1800]
table_radai = [0.0, 0.5, 1.0, 1.5, 2.0]
q1_summary = {
    "table_times_s": q1_times,
    "table_radai_cm": table_radai,
    "temperature_table_c": extract_fixed_table(q1, q1_times, table_radai, "temperature_c"),
    "moisture_table": extract_fixed_table(q1, q1_times, table_radai, "moisture"),
    "biot_heat": H_T * R_FIXED / 0.36,
    "initial_diffusivity_m2_s": float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "biot_mass_initial": H_M * R_FIXED / float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "picard_max": q1["picard_max"],
    "picard_mean": q1["picard_mean"],
    "temperature_bounds_c": [
        float(np.min(q1["temperature_c"])),
        float(np.max(q1["temperature_c"])),
    ],
    "moisture_bounds": [
        float(np.min(q1["moisture"])),
        float(np.max(q1["moisture"])),
    ],
}
write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)

q2 = simulate_fixed(
    environment,

```

```

    property_q23,
    end_time_s=None,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=True,
    stop_interval_s=60.0,
)
save_npz(RESULTS_DIR / "q2" / "fields.npz", q2)
q2_times = [1800, 3600, 5400, 7200, 9000, 10800]
q2_summary = {
    "table_times_h": [time / 3600.0 for time in q2_times],
    "table_radii_cm": table_radii,
    "temperature_table_c": extract_fixed_table(q2, q2_times, table_radii, "temperature_c"),
    "moisture_table": extract_fixed_table(q2, q2_times, table_radii, "moisture"),
    "picard_max": q2["picard_max"],
    "picard_mean": q2["picard_mean"],
    "monotone_profiles": q2["monotone_profiles"],
    "center_is_max": q2["center_is_max"],
}
write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)

q3 = dict(q2)
for key in ("times_s", "temperature_c", "moisture"):
    q3[key] = q2[key][:60].copy()
save_npz(RESULTS_DIR / "q3" / "fields.npz", q3)
q3_end = float(q3["threshold_time_s"])
q3_report_times = list(np.arange(6 * 3600, math.floor(q3_end / (6 * 3600)) * 6 * 3600 + 1,
6 * 3600))
if not q3_report_times or abs(q3_report_times[-1] - q3_end) > 1e-6:
    q3_report_times.append(q3_end)
q3_summary = {
    "drying_time_s": q3_end,
    "drying_time_h": q3_end / 3600.0,
    "output_end_s": q3["last_time_s"],
    "table_times_h": [time / 3600.0 for time in q3_report_times],
    "table_radii_cm": table_radii,
    "moisture_table": extract_fixed_table(q3, q3_report_times, table_radii, "moisture"),
    "center_is_max": q3["center_is_max"],
    "monotone_profiles": q3["monotone_profiles"],
    "picard_max": q3["picard_max"],
    "picard_mean": q3["picard_mean"],
}

q4 = simulate_moving(
    environment,
    radius_history,
    dt_s=1.0,

```

```

        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    save_npz(RESULTS_DIR / "q4" / "fields.npz", q4)
    q4_end = float(q4["threshold_time_s"])
    q4_report_times = list(np.arange(6 * 3600, math.floor(q4_end / (6 * 3600)) * 6 * 3600 + 1,
        6 * 3600))
    if not q4_report_times or abs(q4_report_times[-1] - q4_end) > 1e-6:
        q4_report_times.append(q4_end)
    q4_table_radii = [0.0, 0.5, 1.0]
    q4_summary = {
        "drying_time_s": q4_end,
        "drying_time_h": q4_end / 3600.0,
        "output_end_s": q4["last_time_s"],
        "table_times_h": [time / 3600.0 for time in q4_report_times],
        "table_radii_cm": q4_table_radii,
        "columns": q4_table_radii + ["药材表面"],
        "moisture_table": [
            extract_moving_profile(q4, time, q4_table_radii) for time in q4_report_times
        ],
        "radius_at_end_cm": float(radius_history.values(q4_end)[0] * 100.0),
        "center_is_max": q4["center_is_max"],
        "monotone_profiles": q4["monotone_profiles"],
        "maximum_advection_cfl": q4["maximum_advection_cfl"],
        "picard_max": q4["picard_max"],
        "picard_mean": q4["picard_mean"],
    }
}

if run_convergence:
    q3_dt2 = simulate_fixed(
        environment,
        property_q23,
        end_time_s=None,
        dt_s=2.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    q4_dt2 = simulate_moving(
        environment,
        radius_history,
        dt_s=2.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    q4_stationary = simulate_moving(
        environment,
        radius_history,

```

```

        dt_s=1.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
        include_advection=True,
    )
    q3_summary["time_step_check"] = {
        "dt1_s_time_h": q3_end / 3600.0,
        "dt2_s_time_h": float(q3_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q3_dt2["threshold_time_s"]) - q3_end
        ) / float(q3_dt2["threshold_time_s"]),
    }
    q4_summary["time_step_check"] = {
        "dt1_s_time_h": q4_end / 3600.0,
        "dt2_s_time_h": float(q4_dt2["threshold_time_s"]) / 3600.0,
        "relative_difference": abs(
            float(q4_dt2["threshold_time_s"]) - q4_end
        ) / float(q4_dt2["threshold_time_s"]),
    }
    q4_summary["moving_term_comparison"] = {
        "material_model_h": q4_end / 3600.0,
        "stationary_spatial_model_h": float(q4_stationary["threshold_time_s"]) / 3600.0,
        "difference_h": (
            float(q4_stationary["threshold_time_s"]) - q4_end
        ) / 3600.0,
    }

for summary, simulation in [(q1_summary, q1), (q2_summary, q2), (q3_summary, q3),
(q4_summary, q4)]:
    summary['numerics'] = {'nodes': NODES, 'grid_power': GRID_POWER, 'dt_s': 1.0,
'output_nodes': OUTPUT_NODES}
    summary['balance'] = simulation['balance']
    summary['balance']['cumulative_mass_equation_residual'] =
abs(sum(simulation['balance'][key] for key in
['moisture_change', 'boundary_loss', 'relative_transport']))
    q2_summary['output_end_s'] = q2['last_time_s']
    q2_summary['drying_time_h'] = q2['threshold_time_s'] / 3600.0
    q3_summary['shared_trajectory_with_q2'] = bool(np.array_equal(q3['moisture'],
q2['moisture'][:60]))
    q4_summary['velocity_model'] = q4['velocity_model']
    write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)
    write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)
    write_summary(RESULTS_DIR / "q3" / "summary.json", q3_summary)
    write_summary(RESULTS_DIR / "q4" / "summary.json", q4_summary)

print(json.dumps(
    {

```

```

        "q1_surface_temperature_1800_c": q1_summary["temperature_table_c"][-1][-1],
        "q1_center_moisture_1800": q1_summary["moisture_table"][-1][0],
        "q2_center_temperature_3h_c": q2_summary["temperature_table_c"][-1][0],
        "q2_center_moisture_3h": q2_summary["moisture_table"][-1][0],
        "q3_drying_time_h": q3_summary["drying_time_h"],
        "q4_drying_time_h": q4_summary["drying_time_h"],
        "q4_radius_at_end_cm": q4_summary["radius_at_end_cm"],
    },
    ensure_ascii=False,
    indent=2,
))

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-convergence", action="store_true")
    args = parser.parse_args()
    run_all(run_convergence=not args.skip_convergence)

if __name__ == "__main__":
    main()

```